Day 1:

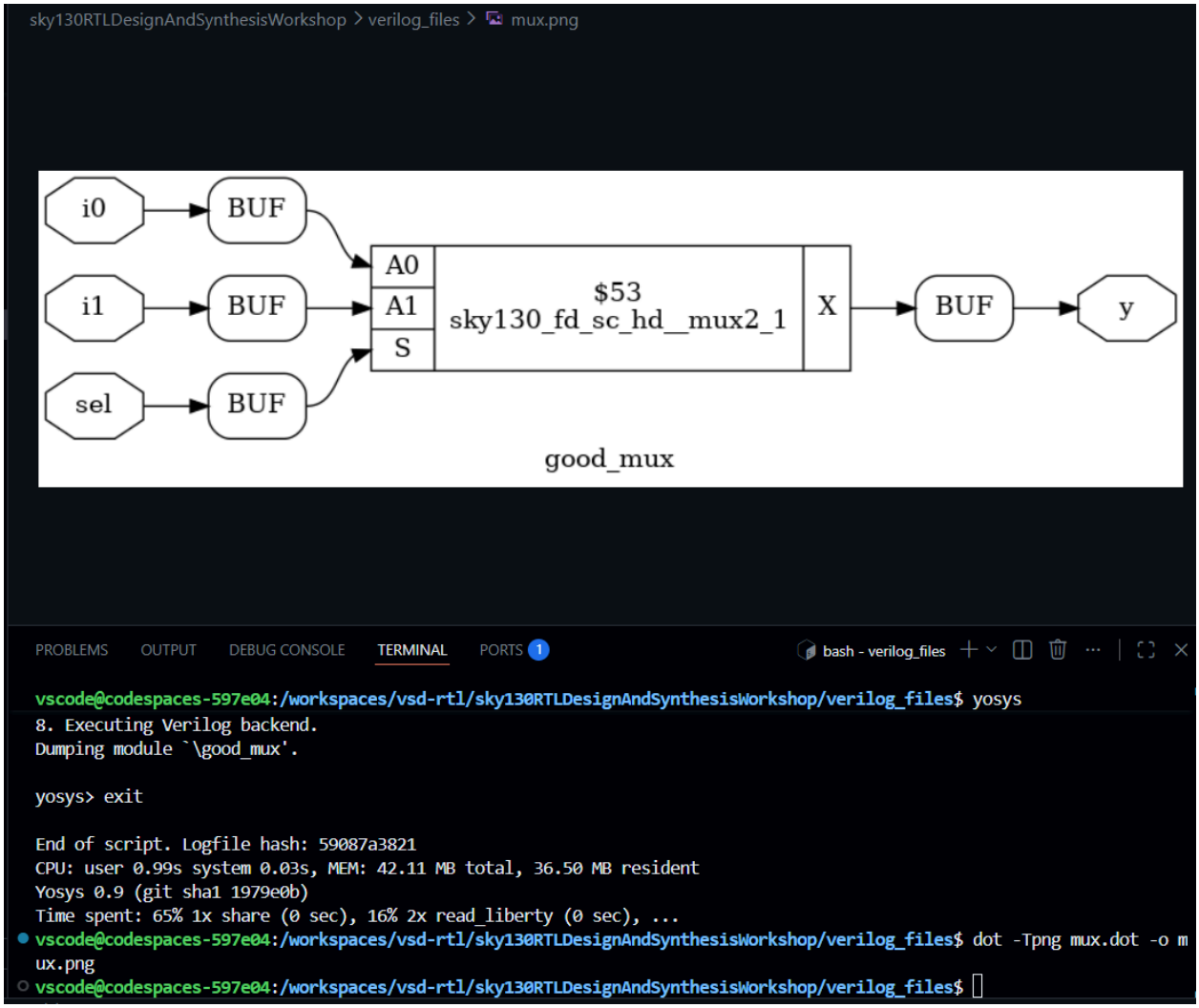
sky130RTLDesignAndSynthesisWorkshop > verilog_files > good_mux_netlist.v

```
1  /* Generated by Yosys 0.9 (git sha1 1979e0b) */
2
3  module good_mux(i0, i1, sel, y);
4    wire _0_;
5    wire _1_;
6    wire _2_;
7    wire _3_;
8    input i0;
9    input i1;
10   input sel;
11   output y;
12   sky130_fd_sc_hd__mux2_1 _4_ (
13     .A0(_0_),
14     .A1(_1_),
15     .S(_2_),
16     .X(_3_)
17   );
18   assign _0_ = i0;
19   assign _1_ = i1;
20   assign _2_ = sel;
21   assign y = _3_;
22 endmodule
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1 yosys - verilog_files

```
vscode@codespaces-597e04:/workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop/verilog_files$ yosys

yosys> write_verilog good_mux_netlist.v

7. Executing Verilog backend.
Dumping module `\good_mux'.

yosys> write_verilog -noattr good_mux_netlist.v

8. Executing Verilog backend.
Dumping module `\good_mux'.

yosys>
```

This image shows the synthesized gate-level netlist of the 2:1 multiplexer generated by Yosys after technology mapping. The module good_mux includes input ports (i0, i1, and sel) and output port (y), along with internal wires used for signal connections. The standard cell sky130_fd_sc_hd__mux2_1 is instantiated, where the inputs are mapped to pins A0, A1, and S, and the output is taken from pin X. The assign statements connect the external inputs to internal wires and route the final output to y. This netlist representation verifies that the behavioral RTL code has been successfully translated into a structural hardware description using standard cells from the SKY130 library.



This image represents the synthesized gate-level diagram of the 2:1 multiplexer after technology mapping using the SKY130 standard cell library. The input signals (i0, i1, and sel) are first passed through buffer cells to improve signal strength and drive capability. These buffered inputs are then connected to the standard cell sky130_fd_sc_hd__mux2_1, where the actual multiplexing operation takes place. Based on the value of the select line (sel), either i0 or i1 is chosen and forwarded to the output. The output is further passed through a buffer before being assigned to y. This diagram confirms that the RTL design has been successfully converted into an optimized hardware implementation using standard cells.

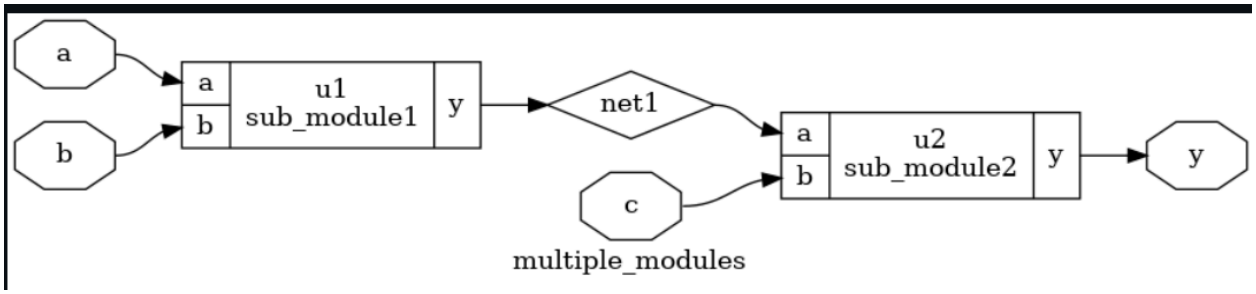
Day 2:

```
yosys> read_liberty -lib ../lib/sky130_fd_sc_hd__tt_025C_1v80.lib
1. Executing Liberty frontend.
Imported 418 cell types from liberty file.

yosys> read_verilog multiple_modules.v

2. Executing Verilog-2005 frontend: multiple_modules.v
Parsing Verilog input from `multiple_modules.v' to AST representation.
Generating RTLIL representation for module `\sub_module2'.
Generating RTLIL representation for module `\sub_module1'.
Generating RTLIL representation for module `\multiple_modules'.
Successfully finished Verilog frontend.

yosys> synth -top multiple_modules

3. Executing SYNTH pass.

3.1. Executing HIERARCHY pass (managing design hierarchy).

3.1.1. Analyzing design hierarchy..
Top module:  \multiple_modules
Used module:     \sub_module2
Used module:     \sub_module1

3.1.2. Analyzing design hierarchy..
Top module:  \multiple_modules
Used module:     \sub_module2
Used module:     \sub_module1
Removed 0 unused modules.

3.2. Executing PROC pass (convert processes to netlists).
```

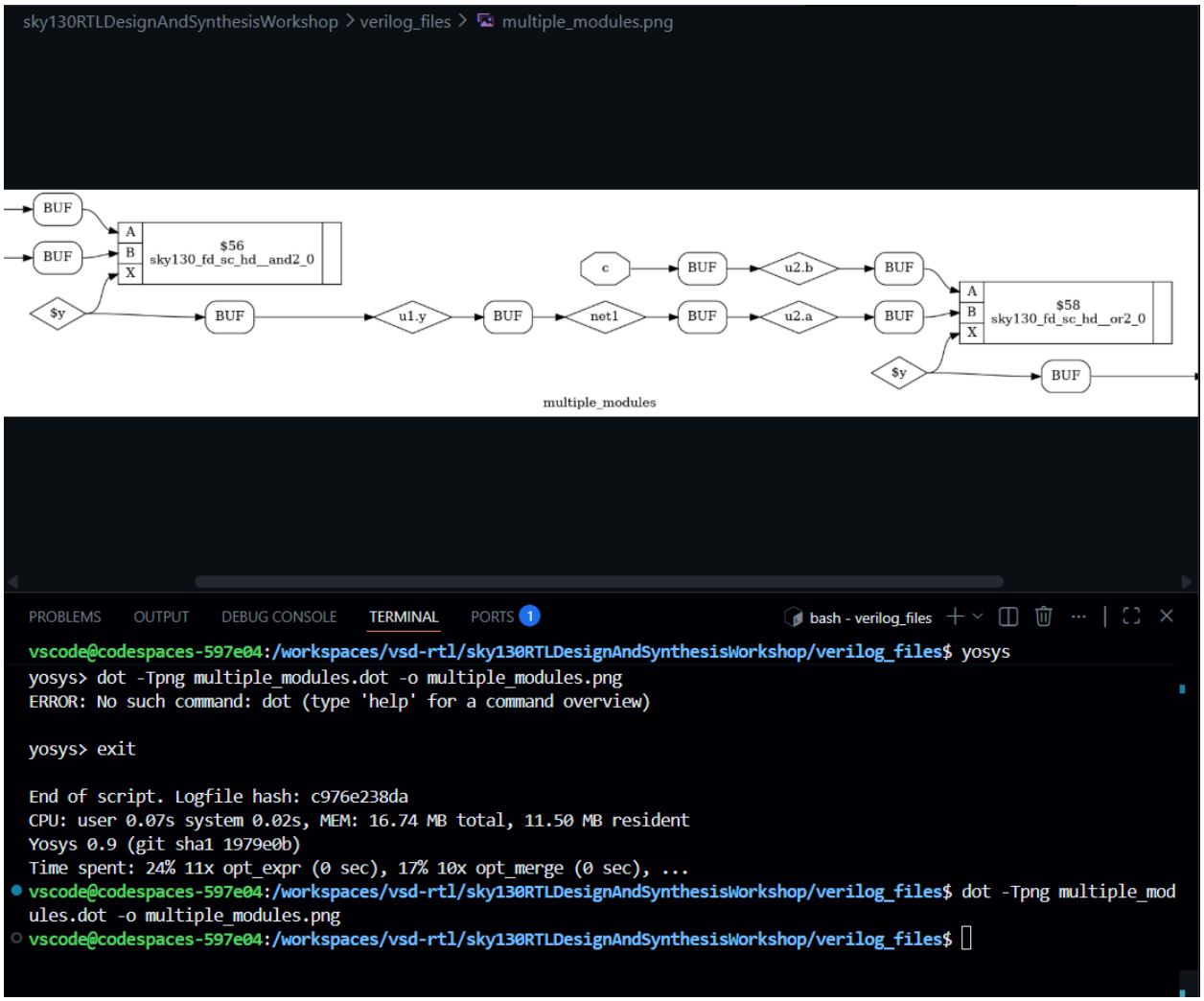
```
=== sub_module2 ===

   Number of wires:                 3
   Number of wire bits:             3
   Number of public wires:          3
   Number of public wire bits:      3
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 1
     $_OR_                          1

=== design hierarchy ===

   multiple_modules                 1
     sub_module1                    1
     sub_module2                    1

   Number of wires:                11
   Number of wire bits:            11
   Number of public wires:         11
   Number of public wire bits:     11
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                 2
     $_AND_                         1
     $_OR_                          1

3.27. Executing CHECK pass (checking for obvious problems).
checking module multiple_modules..
checking module sub_module1..
checking module sub_module2..
found and reported 0 problems.

yosys>
```

```
      Number of processes:                0
      Number of cells:                    2
        $_AND_                            1
        $_OR_                             1

  3.27. Executing CHECK pass (checking for obvious problems).
  checking module multiple_modules..
  checking module sub_module1..
  checking module sub_module2..
  found and reported 0 problems.

  yosys> abc -liberty ../lib/sky130_fd_sc_hd__tt_025C_1v80.lib

  4. Executing ABC pass (technology mapping using ABC).

  4.1. Extracting gate netlist of module `\multiple_modules' to `<abc-temp-dir>/input.blif'..
  Extracted 0 gates and 0 wires to a netlist network with 0 inputs and 0 outputs.
  Don't call ABC as there is nothing to map.
  Removing temp directory.

  4.2. Extracting gate netlist of module `\sub_module1' to `<abc-temp-dir>/input.blif'..
  Extracted 1 gates and 3 wires to a netlist network with 2 inputs and 1 outputs.

  4.2.1. Executing ABC.
  Running ABC command: berkeley-abc -s -f <abc-temp-dir>/abc.script 2>&1
  ABC: ABC command line: "source <abc-temp-dir>/abc.script".
  ABC:
  ABC: + read_blif <abc-temp-dir>/input.blif
  ABC: + read_lib -w /workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop/verilog_files/../lib/sky1
  5C_1v80.lib
  ABC: Parsing finished successfully.  Parsing time =     0.14 sec
  ABC: Scl_LibertyReadGenlib() skipped cell "sky130_fd_sc_hd__decap_12" without logic function.
```

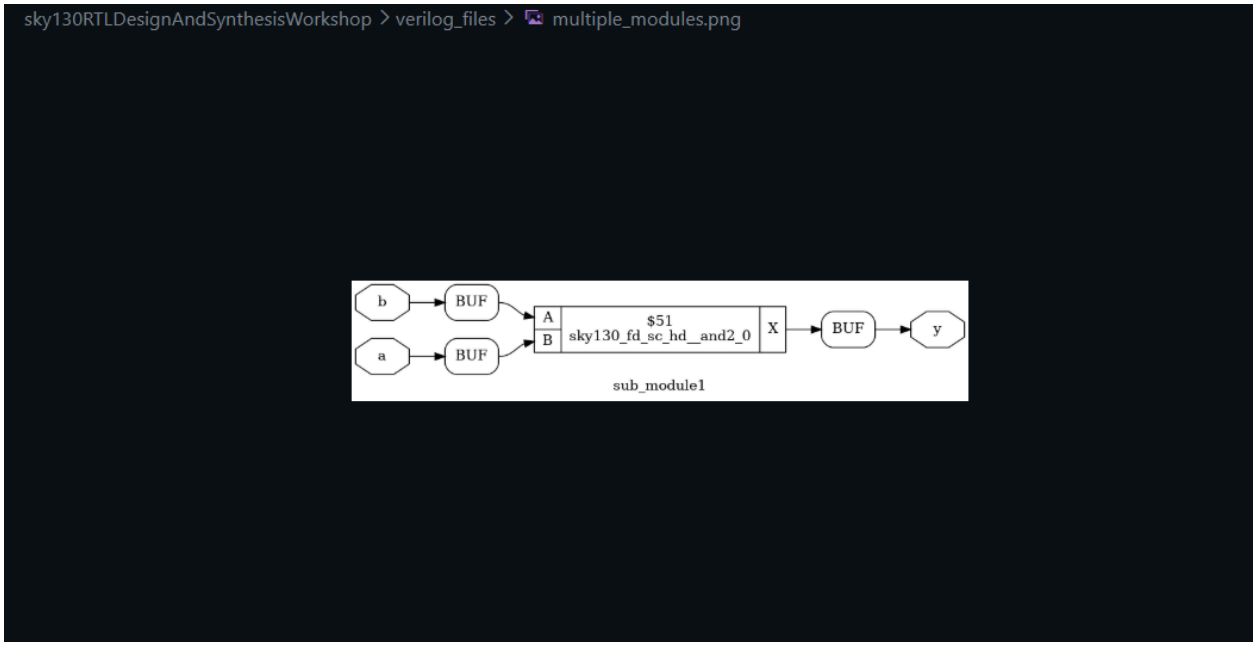
```
    Time =     0.15 sec
ABC: Memory =   15.77 MB. Time =     0.15 sec
ABC: Warning: Detected 9 multi-output gates (for example, "sky130_fd_sc_hd__fa_1").
ABC: + strash
ABC: + ifraig
ABC: + scorr
ABC: Warning: The network is combinational (run "fraig" or "fraig_sweep").
ABC: + dc2
ABC: + dretime
ABC: + retime
ABC: + strash
ABC: + &get -n
ABC: + &dch -f
ABC: + &nf
ABC: + &put
ABC: + write_blif <abc-temp-dir>/output.blif

4.3.2. Re-integrating ABC results.
ABC RESULTS:   sky130_fd_sc_hd__or2_0 cells:       1
ABC RESULTS:        internal signals:       0
ABC RESULTS:           input signals:       2
ABC RESULTS:          output signals:       1
Removing temp directory.

yosys>
```



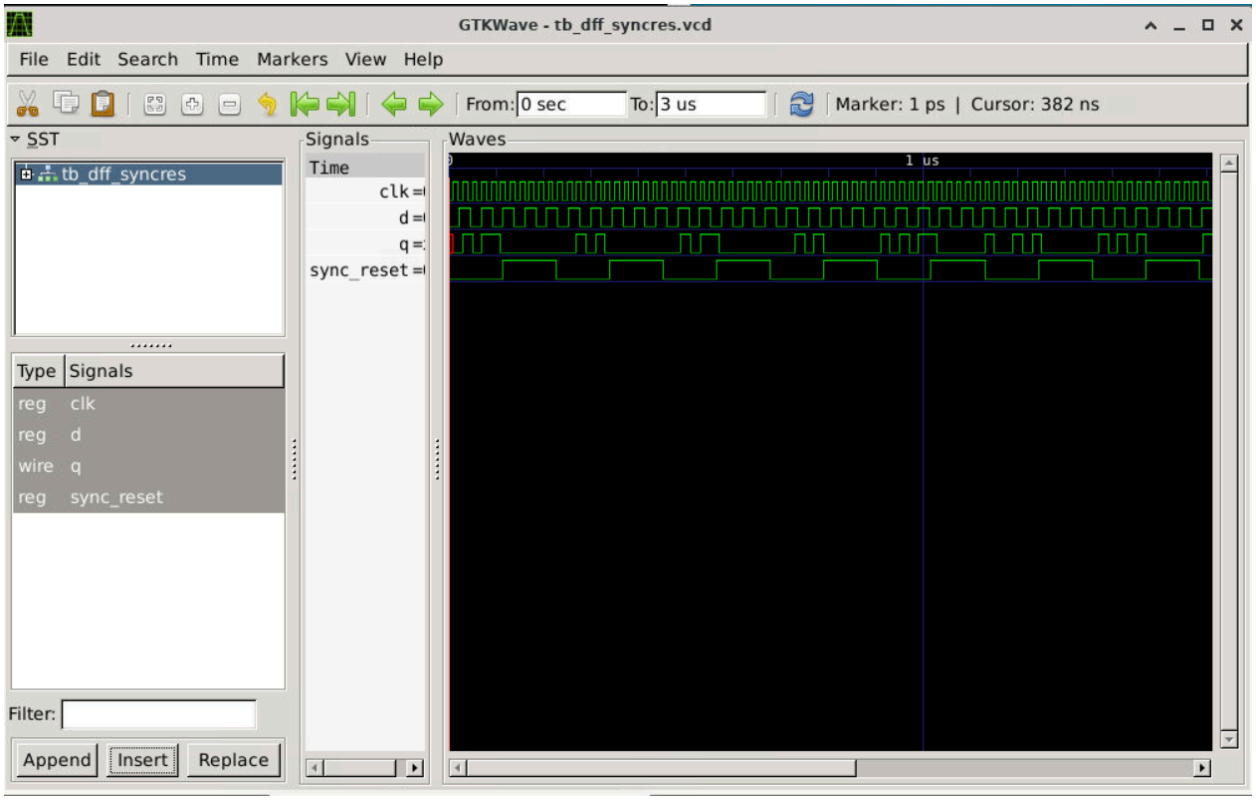
This set of outputs and the corresponding diagram represent the hierarchical synthesis and technology mapping of a combinational design consisting of multiple modules using Yosys and the SKY130 standard cell library. The design is organized into a top module multiple_modules, which instantiates two submodules, sub_module1 and sub_module2, demonstrating a clear hierarchical structure. According to the synthesis report, both submodules are purely combinational with no memories or sequential elements, and they utilize basic logic cells such as AND and OR gates. Specifically, sub_module1 performs an AND operation on inputs a and b, producing an intermediate signal (net1), which is then passed as an input to sub_module2 along with another input c. The sub_module2 performs an OR operation to generate the final output y. The CHECK pass confirms that the design is free from errors and structurally correct. During the ABC technology mapping phase, the logical operations are mapped to actual standard cells from the SKY130 library, such as sky130_fd_sc_hd__or2, ensuring that the design is realizable in hardware. The final gate-level diagram visually confirms this flow, showing how inputs propagate through interconnected submodules to produce the output, thereby validating that the RTL design has been successfully synthesized, optimized, and mapped into a hardware-level implementation using standard cells.

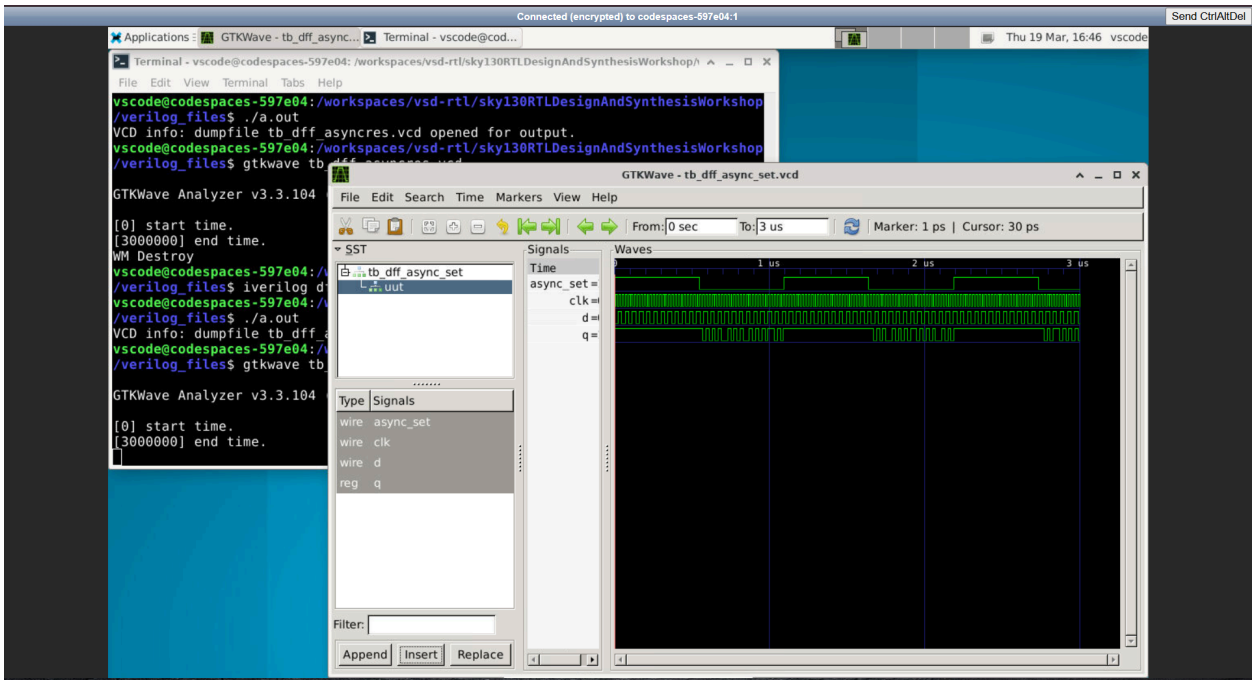


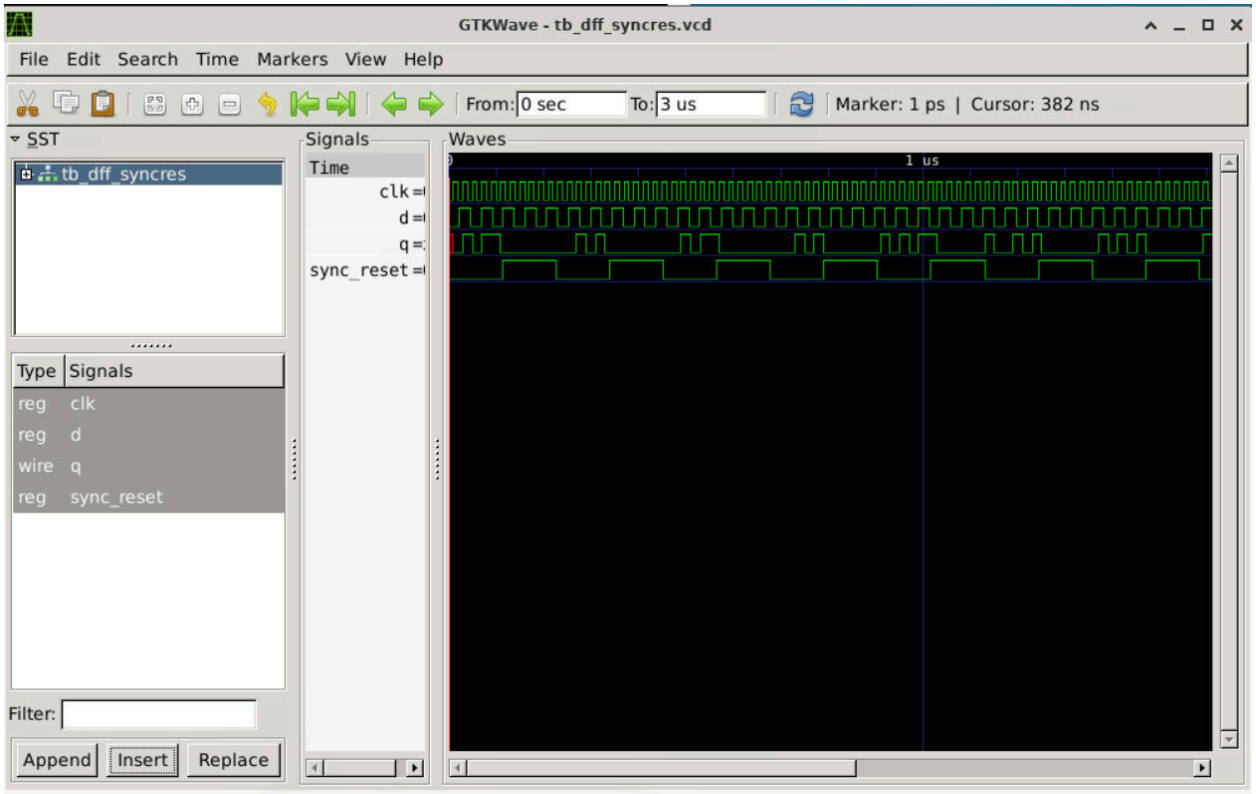
This output and diagram represent the flattened gate-level implementation of the hierarchical design after synthesis and optimization using Yosys and the SKY130 standard cell library. Initially, the design consists of multiple modules (sub_module1 and sub_module2) organized hierarchically, but after applying the flatten command, the hierarchy is removed and all submodules are merged into a single top-level module (multiple_modules). This process simplifies the design by directly connecting all logic elements without module boundaries. The synthesis tool maps the logical operations to standard cells such as sky130_fd_sc_hd__and2 and sky130_fd_sc_hd__or2, which implement the AND and OR functionalities respectively. The inputs are passed through buffer cells to ensure proper signal strength and are then fed into the AND gate, producing an intermediate signal that is routed through additional buffers and connections. This intermediate signal, along with input c, is then applied to the OR gate to generate the final output y. The presence of multiple buffers indicates optimization for signal integrity and fan-out handling. The terminal output confirms successful flattening, removal of unused modules, and generation of the final netlist file, while the diagram visually verifies that the hierarchical design has been converted into a fully flattened, optimized gate-level hardware implementation using standard cells.

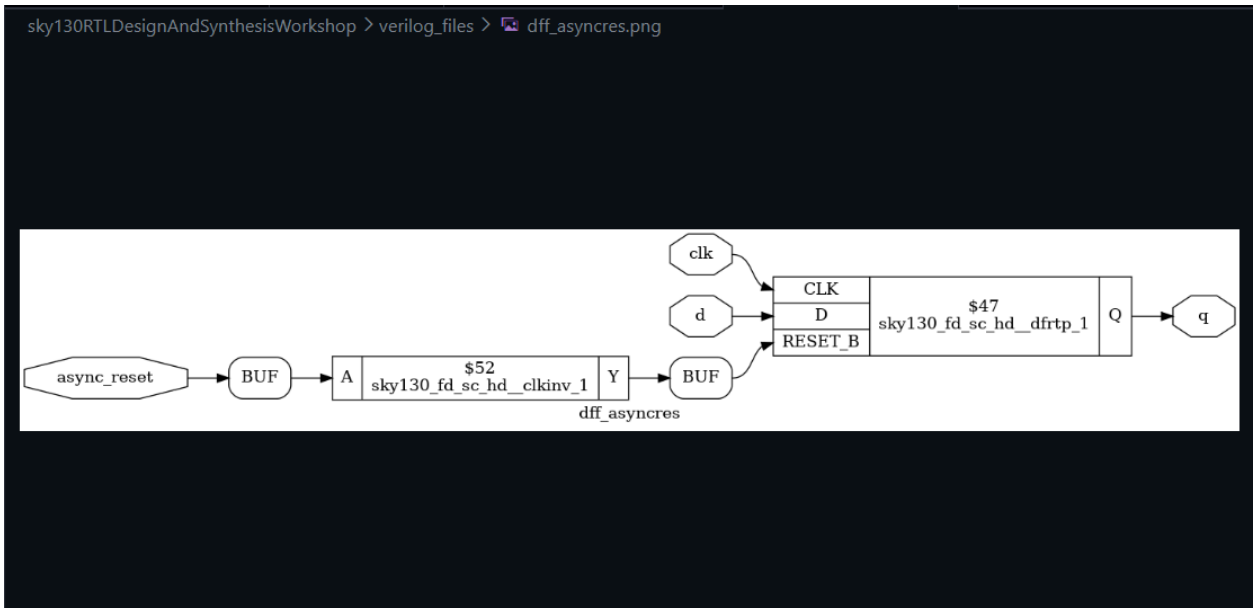


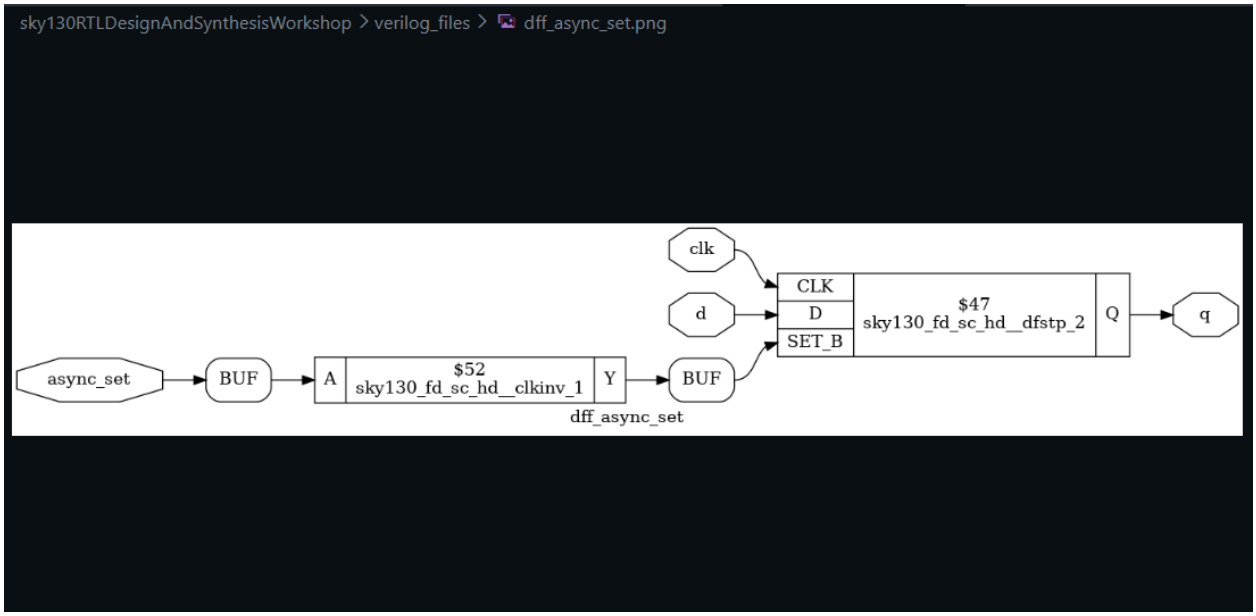
This diagram represents the synthesized gate-level implementation of sub_module1 after technology mapping using the SKY130 standard cell library. The input signals a and b are first passed through buffer cells to improve signal strength and ensure proper drive capability. These buffered inputs are then connected to the standard cell sky130_fd_sc_hd__and2_0, which performs the logical AND operation. The output of this AND gate is taken from pin X and passed through an additional buffer before being assigned to the final output y. This structure confirms that the original RTL description of sub_module1 has been successfully synthesized and mapped into a hardware-level representation using optimized standard cells, with buffers included to maintain signal integrity and timing reliability.

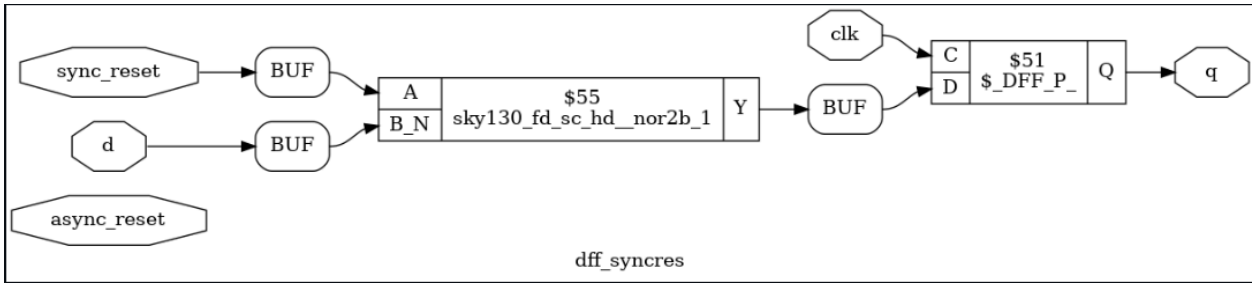

GTKWave - tb_dff_syncres.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 3 us
Marker: 1 ps | Cursor: 382 ns
SST
tb_dff_syncres
Signals
Time
clk
d
q
sync_reset
Waves
1 us
Type Signals
reg clk
reg d
wire q
reg sync_reset
Filter:
Append
Insert
Replace


Connected (encrypted) to codespaces-597e04:1
Send CtrlAltDel
Applications
GTKWave - tb_dff_async...
Terminal - vscode@cod...
Thu 19 Mar, 16:46 vscode
Terminal - vscode@codespaces-597e04: /workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop/
File Edit View Terminal Tabs Help
vscode@codespaces-597e04:/workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop
/verilog_files$ ./a.out
VCD info: dumpfile tb_dff_asyncres.vcd opened for output.
vscode@codespaces-597e04:/workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop
/verilog_files$ gtkwave tb
GTKWave Analyzer v3.3.104
[0] start time.
[3000000] end time.
WM Destroy
/verilog_files$ iverilog d
/verilog_files$ ./a.out
VCD info: dumpfile tb_dff_
/verilog_files$ gtkwave tb
GTKWave Analyzer v3.3.104
[0] start time.
[3000000] end time.
GTKWave - tb_dff_async_set.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 3 us
Marker: 1 ps | Cursor: 30 ps
SST
tb_dff_async_set
uut
Signals
Time
async_set
clk
d
q
Waves
1 us
2 us
3 us
Type Signals
wire async_set
wire clk
wire d
reg q
Filter:
Append
Insert
Replace

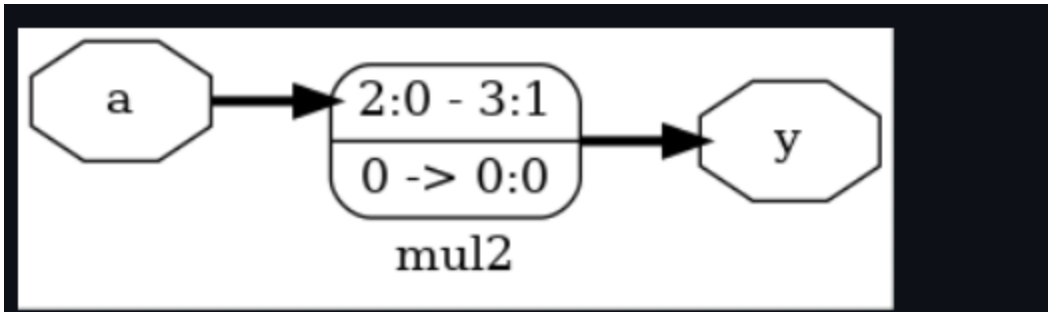

GTKWave - tb_dff_syncres.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 3 us
Marker: 1 ps | Cursor: 382 ns
SST
tb_dff_syncres
Type Signals
reg clk
reg d
wire q
reg sync_reset
Filter:
Append Insert Replace
Signals
Time
clk
d
q
sync_reset
Waves
1 us

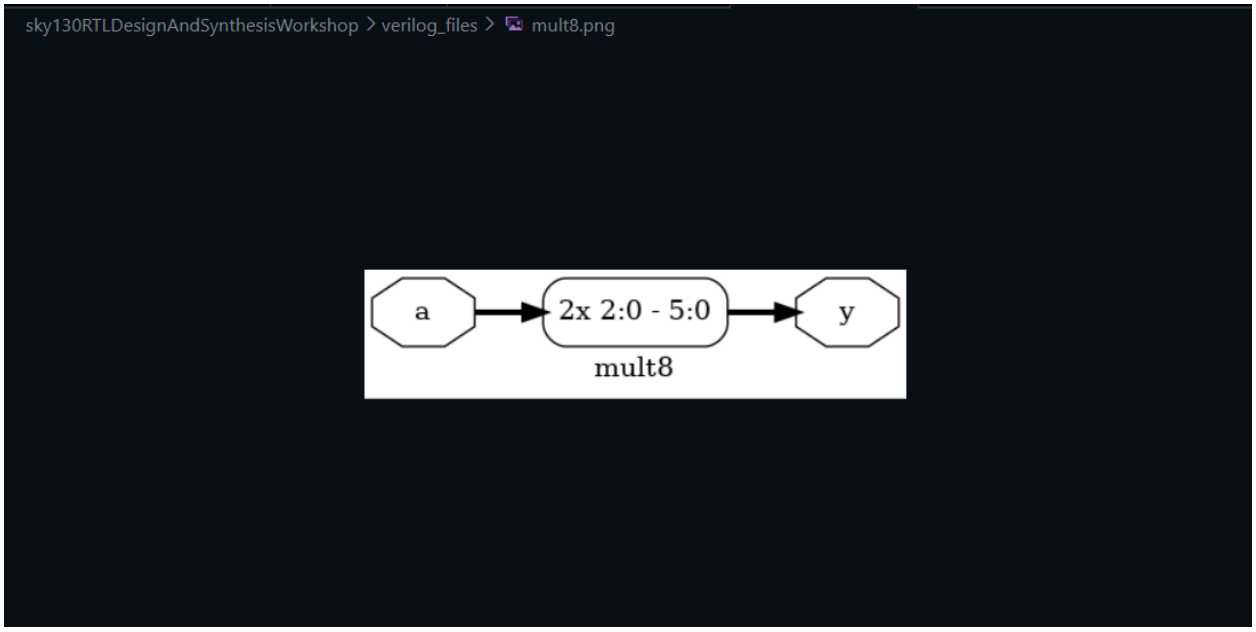

sky130RTLDesignAndSynthesisWorkshop > verilog_files > dff_asyncres.png
clk
d
async_reset
BUF
A
$52
sky130_fd_sc_hd__clkinv_1
Y
BUF
CLK
D
RESET_B
$47
sky130_fd_sc_hd__dfrtp_1
Q
q
dff_asyncres





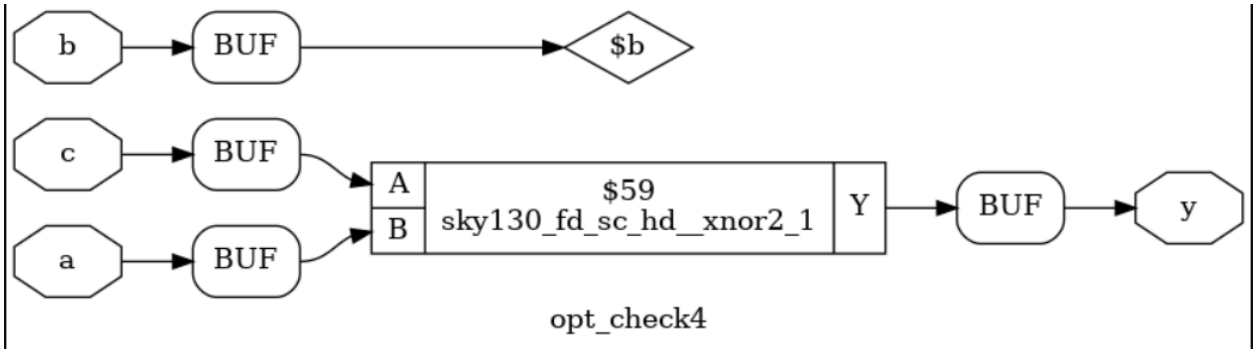
These images collectively represent the simulation and synthesized gate-level implementations of different D Flip-Flop (DFF) designs with asynchronous reset, asynchronous set, and synchronous reset using the SKY130 standard cell library. The GTKWave outputs show the timing waveforms of signals such as clk, d, async_reset, async_set, and output q, verifying the functional correctness of each design. In the asynchronous reset design, the output q is immediately forced to logic 0 whenever the reset signal is asserted, independent of the clock, which is evident from the waveform. Similarly, in the asynchronous set design, the output is forced to logic 1 when the set signal is active, regardless of the clock edge. In contrast, the synchronous reset design updates the output only on the active clock edge, showing that reset behavior is clock-dependent. The corresponding gate-level diagrams illustrate how these behaviors are implemented using SKY130 standard cells such as sky130_fd_sc_hd__dfrtp_1 (DFF with reset), sky130_fd_sc_hd__dfstp_2 (DFF with set), and basic logic gates like nor2b for synchronous reset logic. Buffers are inserted to ensure proper signal strength and timing reliability, and clock inversion cells (clkinv) are used where required for active-low control signals. Overall, these results confirm that the RTL descriptions of different flip-flop configurations have been successfully simulated, synthesized, and mapped into accurate hardware-level implementations, demonstrating both functional correctness through waveforms and structural realization through standard cell-based gate-level designs.

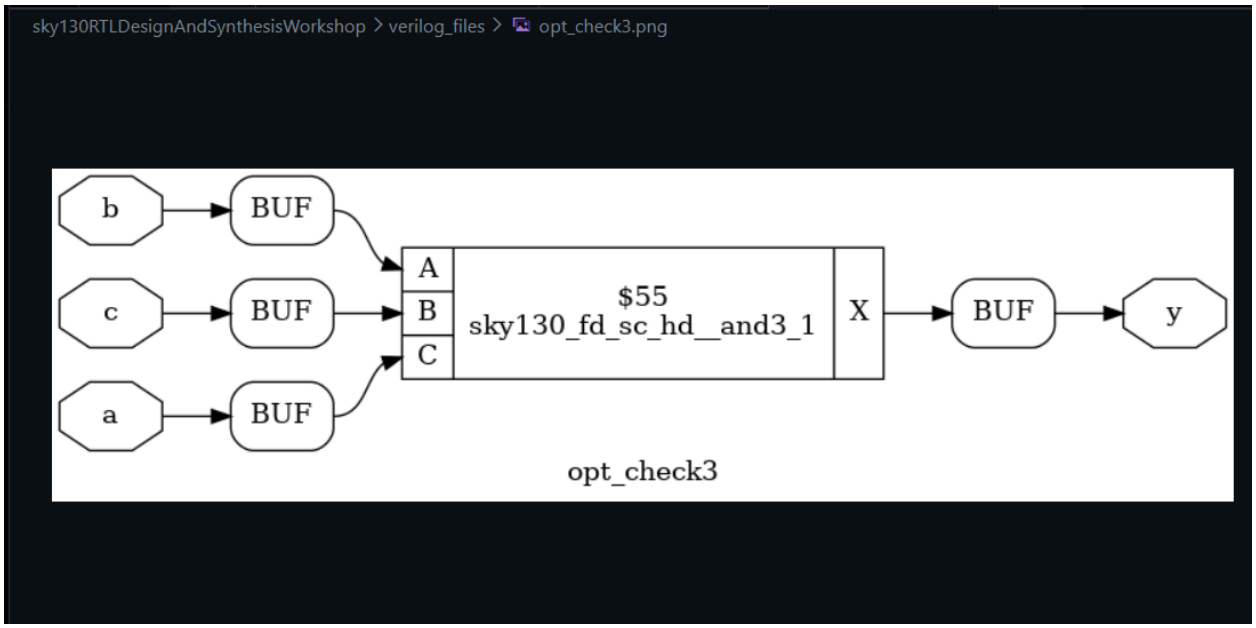


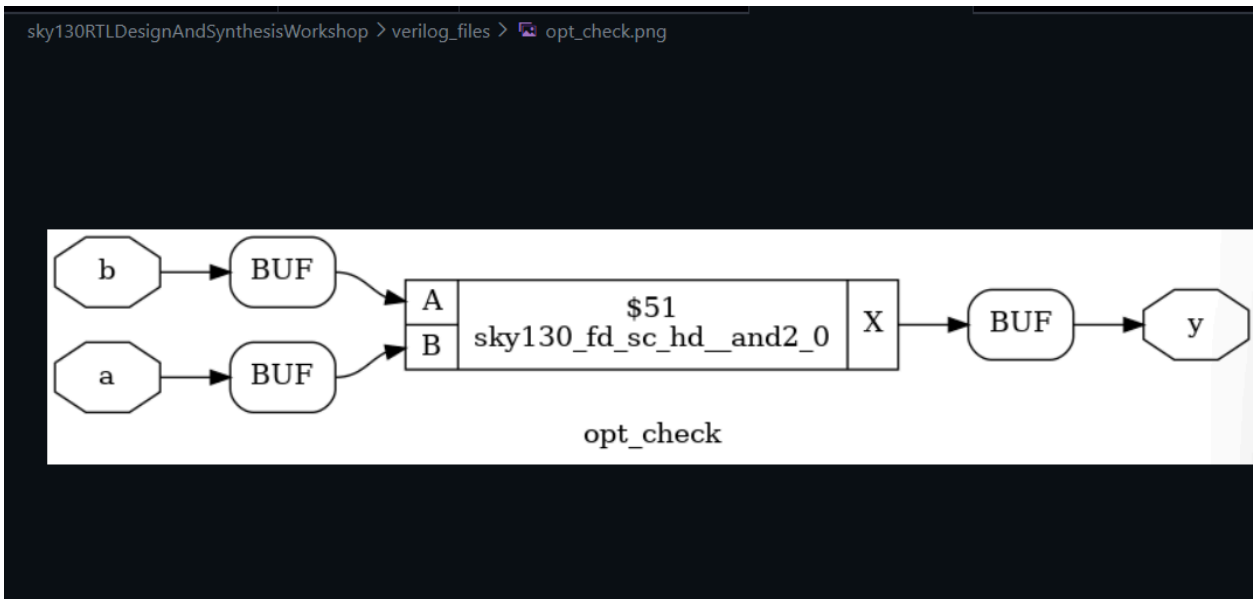


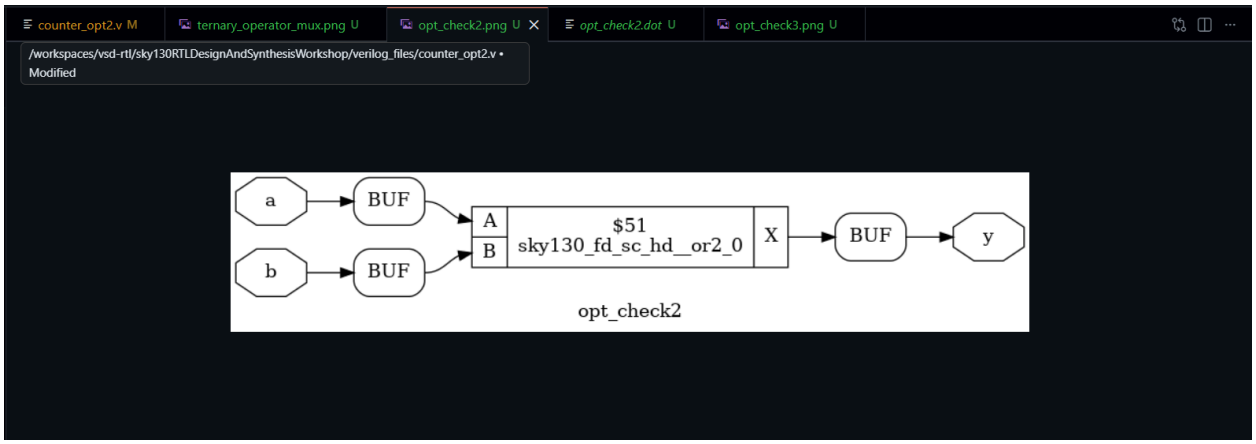
These diagrams represent the synthesized implementations of simple multiplier-based operations generated from RTL descriptions using Yosys. In the first case (mul2), the diagram shows that the input a is effectively scaled by a factor of 2, which is implemented as a bit-level transformation rather than using a dedicated multiplier hardware block. This is evident from the notation indicating bit slicing and shifting (e.g., mapping bits 2:0 to 3:1), meaning the operation is realized as a left shift, which is a hardware-efficient way to perform multiplication by 2. Similarly, in the second case (mult8), the input a is multiplied by 8, which is again implemented using a left shift operation (shifting by 3 positions), as indicated by the transformation from input bit range to a higher output bit range (2:0 to 5:0). These implementations avoid complex multiplier circuits and instead use simple wiring and bit manipulation, making them highly area-efficient and fast. The output y in both cases directly reflects the shifted version of the input, confirming that the synthesis tool has optimized constant multiplications into shift operations. This demonstrates how RTL arithmetic operations are intelligently mapped into efficient hardware structures during synthesis.

Day 3:


b
BUF
$b
c
BUF
A
B
$59
sky130_fd_sc_hd__xnor2_1
Y
BUF
y
a
BUF
opt_check4


sky130RTLDesignAndSynthesisWorkshop > verilog_files > opt_check3.png
b
BUF
A
B
C
$55
sky130_fd_sc_hd__and3_1
X
BUF
y
c
BUF
a
BUF
opt_check3


sky130RTLDesignAndSynthesisWorkshop > verilog_files > opt_check.png
b
BUF
A
B
$51
sky130_fd_sc_hd__and2_0
X
BUF
y
a
BUF
opt_check

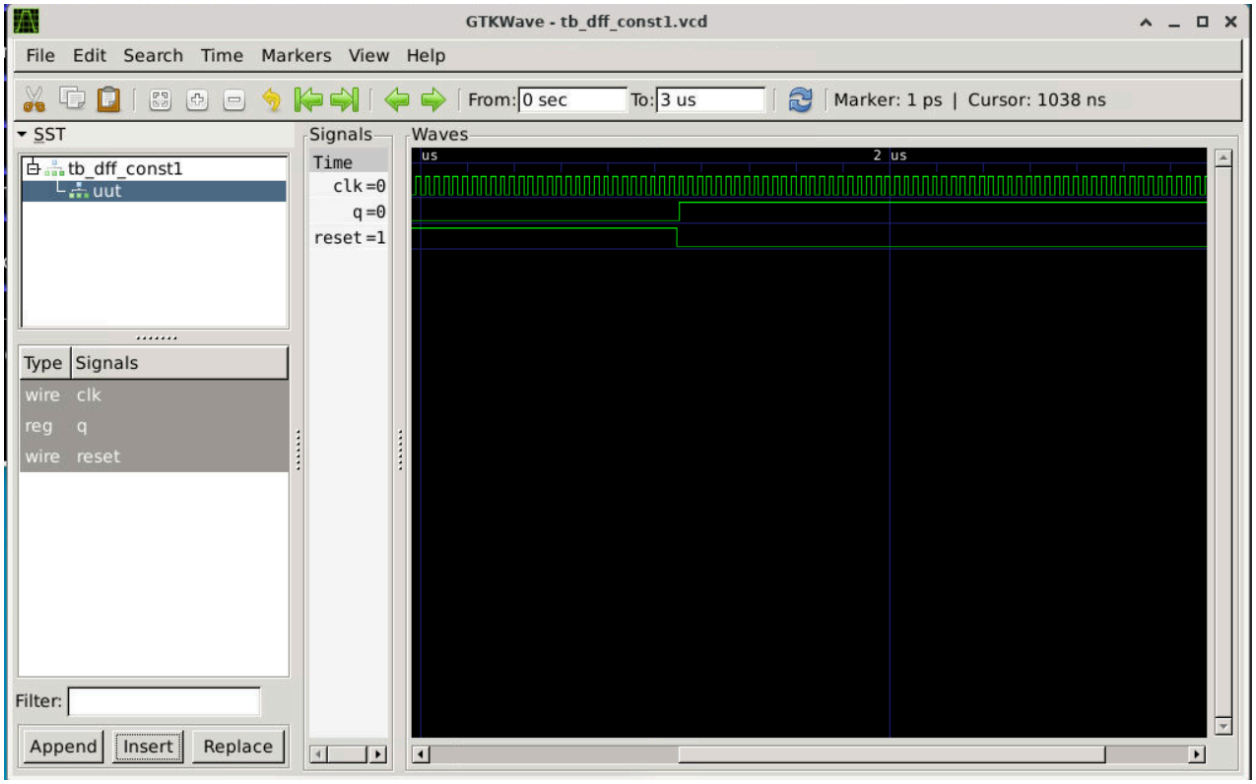


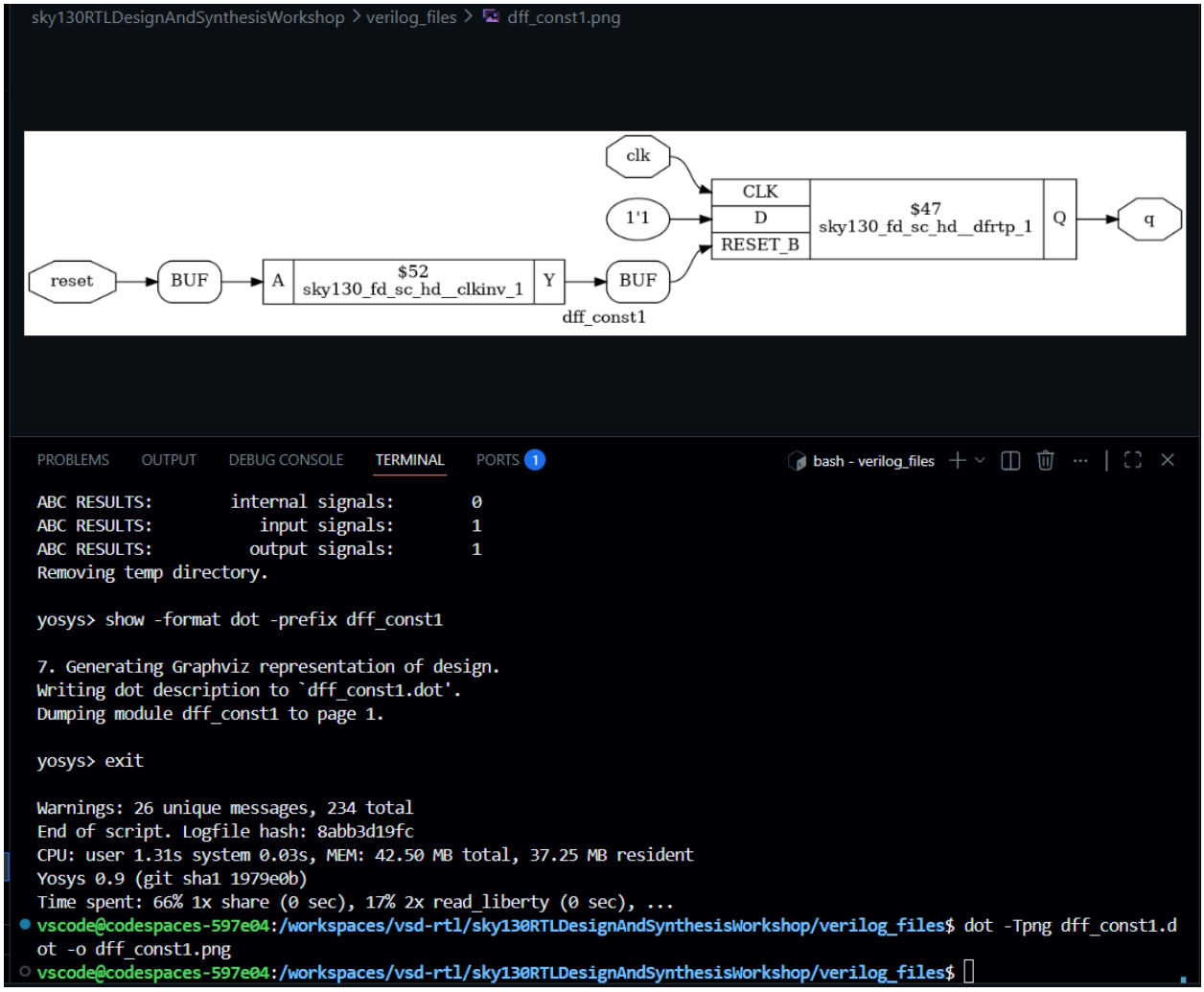
opt_check (2-Input AND): This module establishes the baseline for simple combinational logic. The synthesis tool maps the logic to a sky130_fd_sc_hd__and2_0 cell, where the "0" indicates a base drive strength. The tool automatically inserts buffers on the inputs (a, b) and the output (y) to ensure signal transition speeds meet the library's requirements.

opt_check2 (2-Input OR): Functioning as a direct counterpart to the first module, this design implements an OR function. It utilizes a sky130_fd_sc_hd__or2_0 cell. Despite the change in logic type, the synthesis structure remains consistent, employing the same high-density (hd) standard cell architecture and buffering strategy to maintain electrical integrity across the netlist.

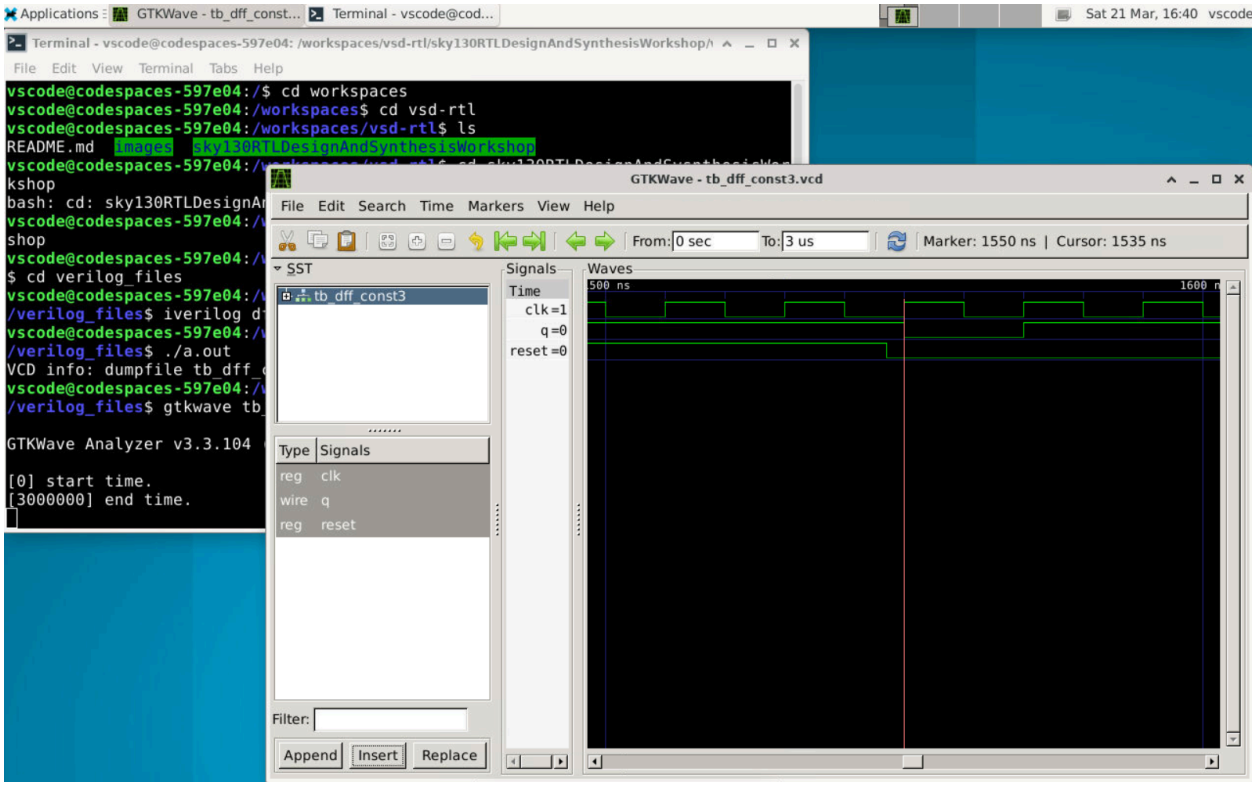
opt_check3 (3-Input AND): This module demonstrates logic collapsing and drive strength optimization. Rather than using a chain of 2-input gates, the tool selects a single sky130_fd_sc_hd__and3_1 cell. The shift to a "drive strength 1" variant suggests that this specific gate is designed to drive a larger capacitive load or a longer wire path compared to the previous modules.

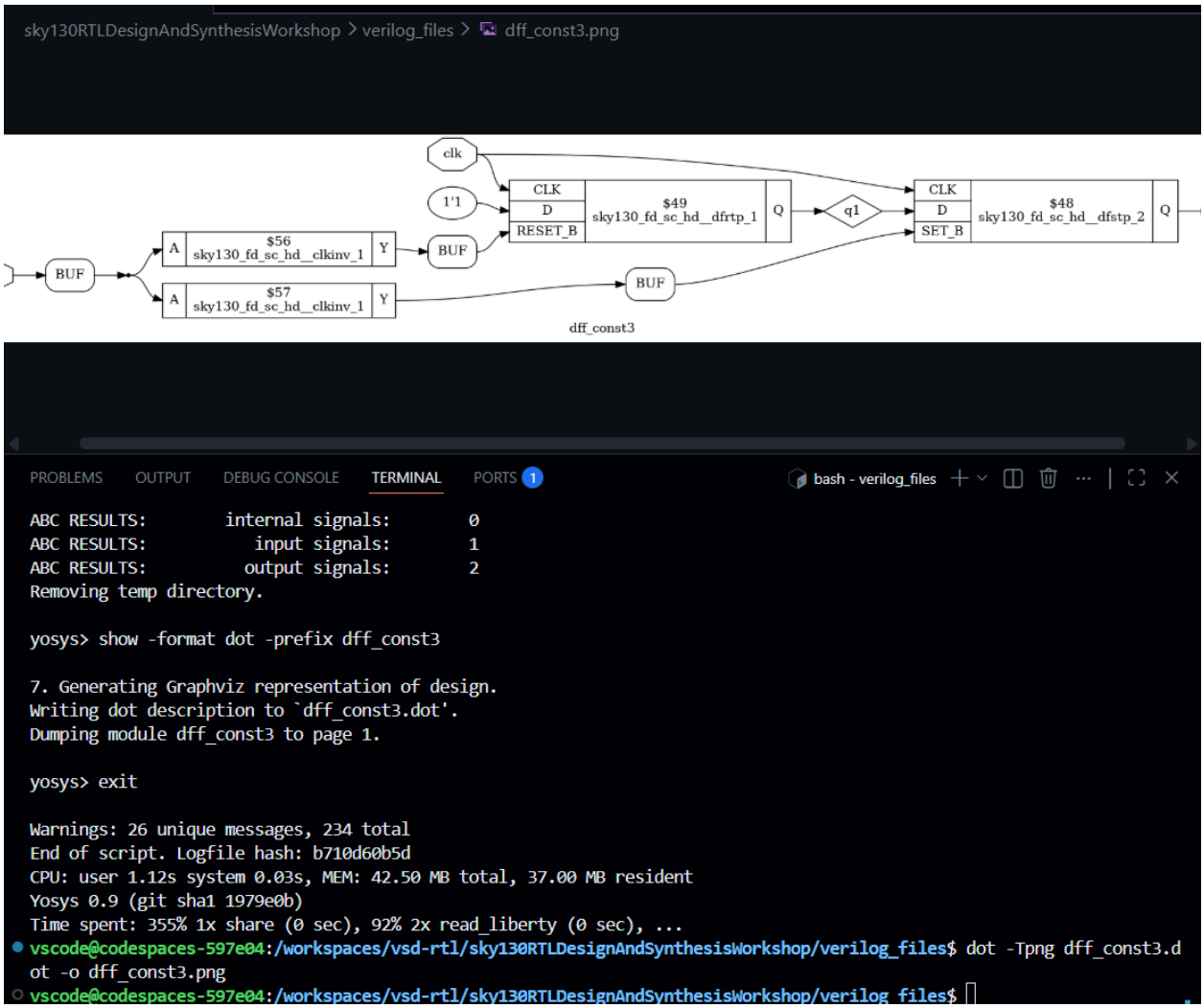
opt_check4 (XNOR and Logic Isolation): This final example highlights how the tool manages complex gates and unused inputs. The output y is driven by a sky130_fd_sc_hd__xnor2_1 cell using inputs a and c. Interestingly, input b is buffered but terminated at a net labeled $b without connecting to the output logic, illustrating how synthesis tools isolate redundant or unused ports while still accounting for them in the physical design space.


GTKWave - tb_dff_const1.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 3 us
Marker: 1 ps | Cursor: 1038 ns
SST
tb_dff_const1
uut
Type Signals
wire clk
reg q
wire reset
Filter:
Append Insert Replace
Signals
Time
clk =0
q =0
reset =1
Waves
us
2 us



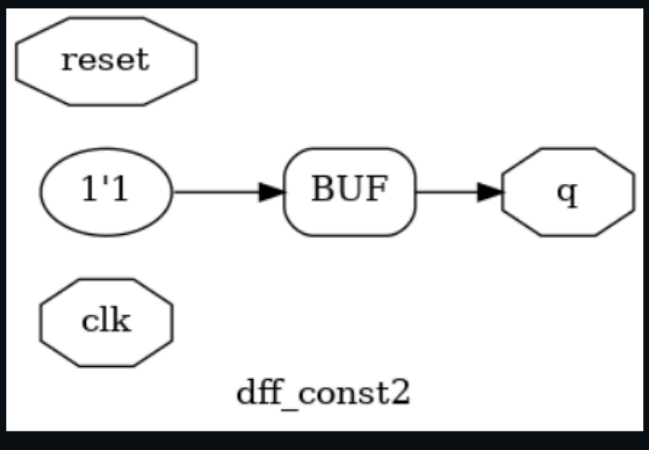
The GTKWave simulation of the tb_dff_const1 testbench confirms the timing behavior of the circuit, where the output q transitions to a high state once the active-low reset signal is de-asserted, remaining constant thereafter. This behavior is physically realized in the synthesized netlist through a sky130_fd_sc_hd__dfrtp_1 cell, a specialized D Flip-Flop where the D input is tied to a constant logic 1'1. To ensure the correct reset polarity, the synthesis tool has mapped the reset signal through a sky130_fd_sc_hd__clkinv_1 clock inverter before it reaches the RESET_B pin. The accompanying terminal output details the Yosys workflow, showing the conversion of the design into a Graphviz dot file and its subsequent rendering into a visual schematic, completing the bridge between functional simulation and library-specific hardware mapping.


Applications
GTKWave - tb_dff_const...
Terminal - vscode@cod...
Sat 21 Mar, 16:40 vscode
Terminal - vscode@codespaces-597e04: /workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop/
File Edit View Terminal Tabs Help
vscode@codespaces-597e04:/$ cd workspaces
vscode@codespaces-597e04:/workspaces$ cd vsd-rtl
vscode@codespaces-597e04:/workspaces/vsd-rtl$ ls
README.md images sky130RTLDesignAndSynthesisWorkshop
kshop
bash: cd: sky130RTLDesignA
shop
$ cd verilog_files
/verilog_files$ iverilog d
/verilog_files$ ./a.out
VCD info: dumpfile tb_dff_
/verilog_files$ gtkwave tb_
GTKWave Analyzer v3.3.104
[0] start time.
[3000000] end time.
GTKWave - tb_dff_const3.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 3 us
Marker: 1550 ns | Cursor: 1535 ns
SST
tb_dff_const3
Type Signals
reg clk
wire q
reg reset
Filter:
Append Insert Replace
Signals
Time
clk=1
q=0
reset=0
Waves
500 ns
1600 n



This set of images shows the simulation and gate-level implementation of the dff_const3 design using the SKY130 standard cell library. The GTKWave waveform verifies the behavior of signals like clk, reset, and output q, showing that when reset is active, the output is forced to a known value, and once reset is released, the flip-flop captures the constant input (1'b1) on the clock edge and holds it. In the synthesized diagram, this behavior is implemented using standard cells such as sky130_fd_sc_hd__dfrtp_1 and sky130_fd_sc_hd__dfstp_2, along with buffers and clock inverter cells to maintain proper signal strength and polarity. Since the input is constant, the logic is simplified, and the output stabilizes accordingly. This confirms correct simulation and efficient hardware mapping of the design.

sky130RTLDesignAndSynthesisWorkshop > verilog_files > dff_const2.png



Applications | GTKWave - tb_dff_const... | Terminal - vscode@cod... | Sat 21 Mar, 16:50 vscode

Terminal - vscode@codespaces-597e04: /workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop/

File Edit View Terminal Tabs Help

```
[0] start time.
[3000000] end time.
WM Destroy
vscode@codespaces-597e04:/workspaces/vsd-rtl/sky130RTLDesignAndSynthesisWorkshop
/verilog_files$ gtkwave tb_dff_const2.vcd

GTKWave Analyzer v3.3.104

[0] start time.
[3000000] end time.
WM Destroy
vscode@codespaces-597e04:/
/verilog_files$ iverilog d
vscode@codespaces-597e04:/
/verilog_files$ ./a.out
VCD info: dumpfile tb_dff_
vscode@codespaces-597e04:/
/verilog_files$ gtkwave tb

GTKWave Analyzer v3.3.104

[0] start time.
[3000000] end time.
```

GTKWave - tb_dff_const4.vcd

File Edit Search Time Markers View Help

From: 0 sec To: 3 us | Marker: 1547 ns | Cursor: 1496400 ps

SST

tb_dff_const4
uut

Type	Signals
wire	clk
reg	q
reg	q1
wire	reset

Filter:

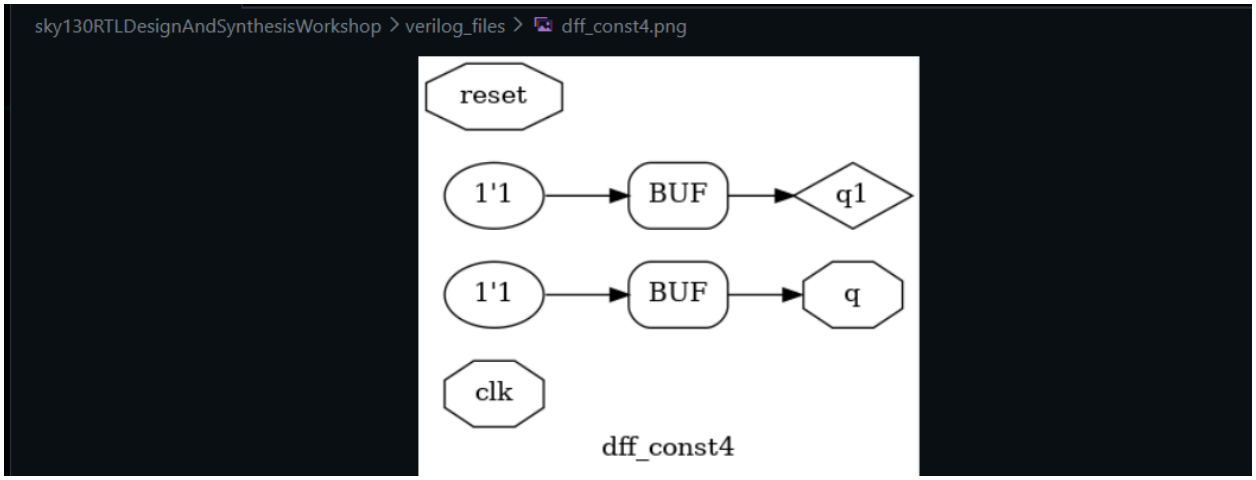
Append Insert Replace

Signals

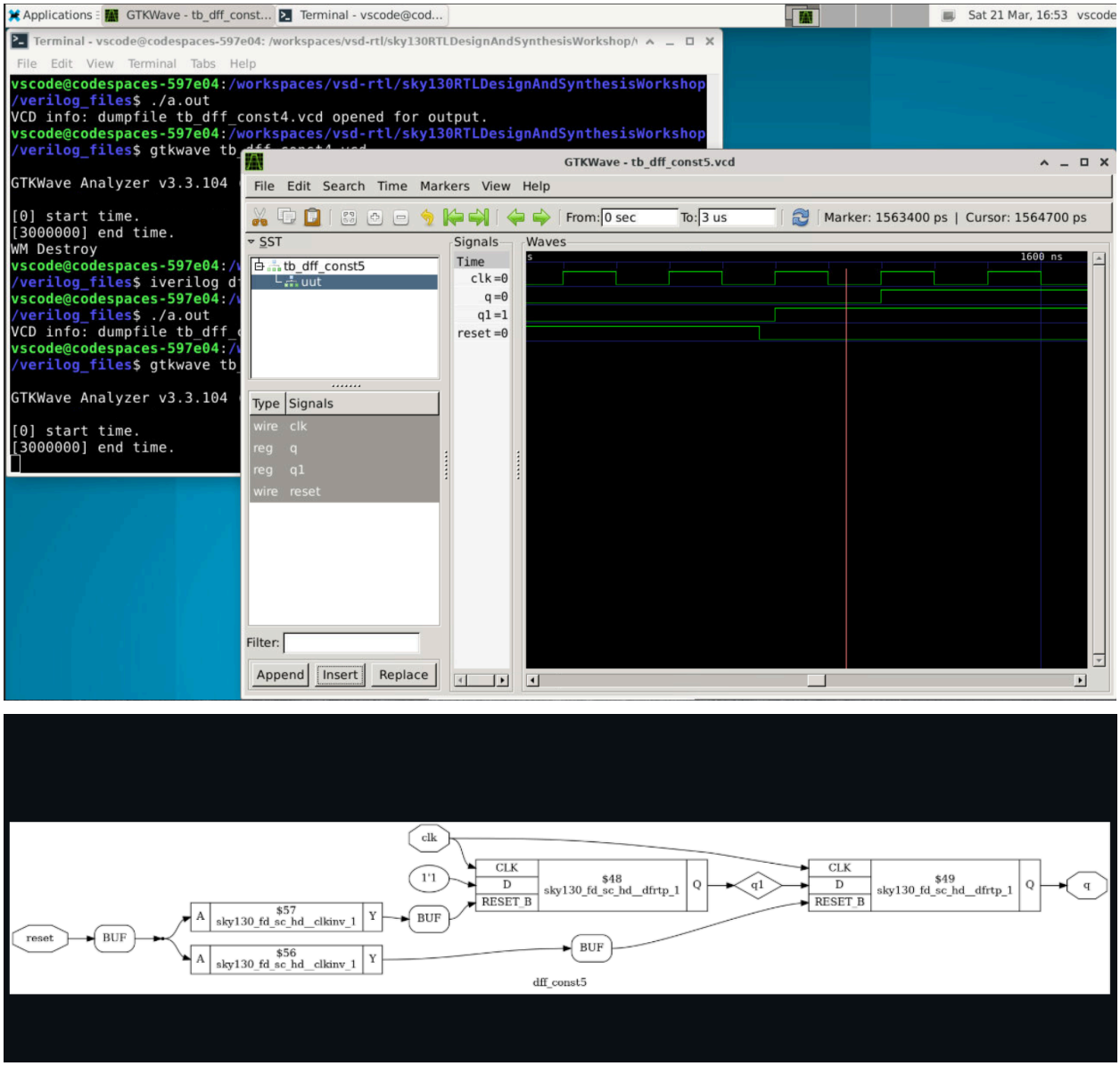
Time
clk=0
q=1
q1=1
reset=0

Waves

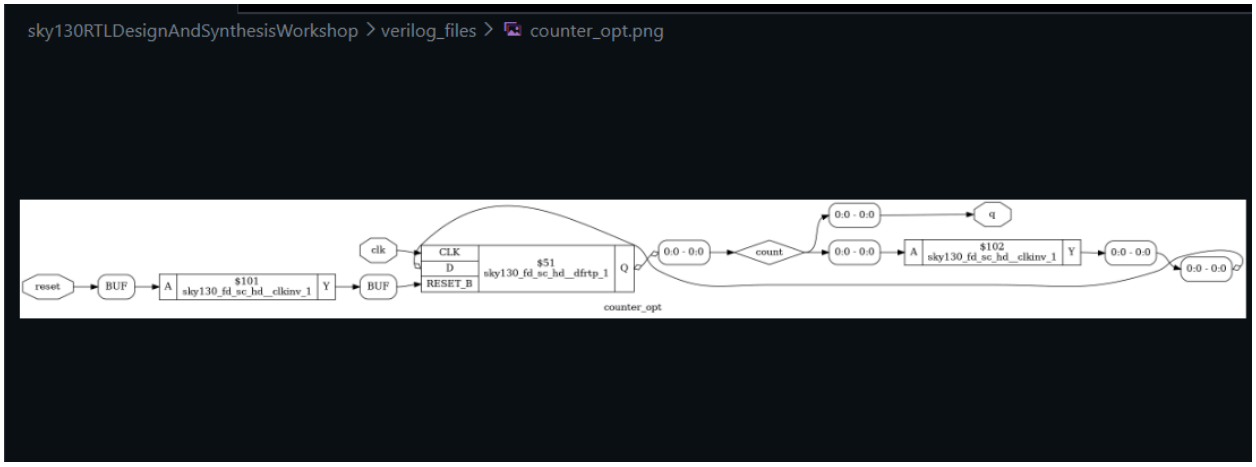
1500 ns 1600 ns

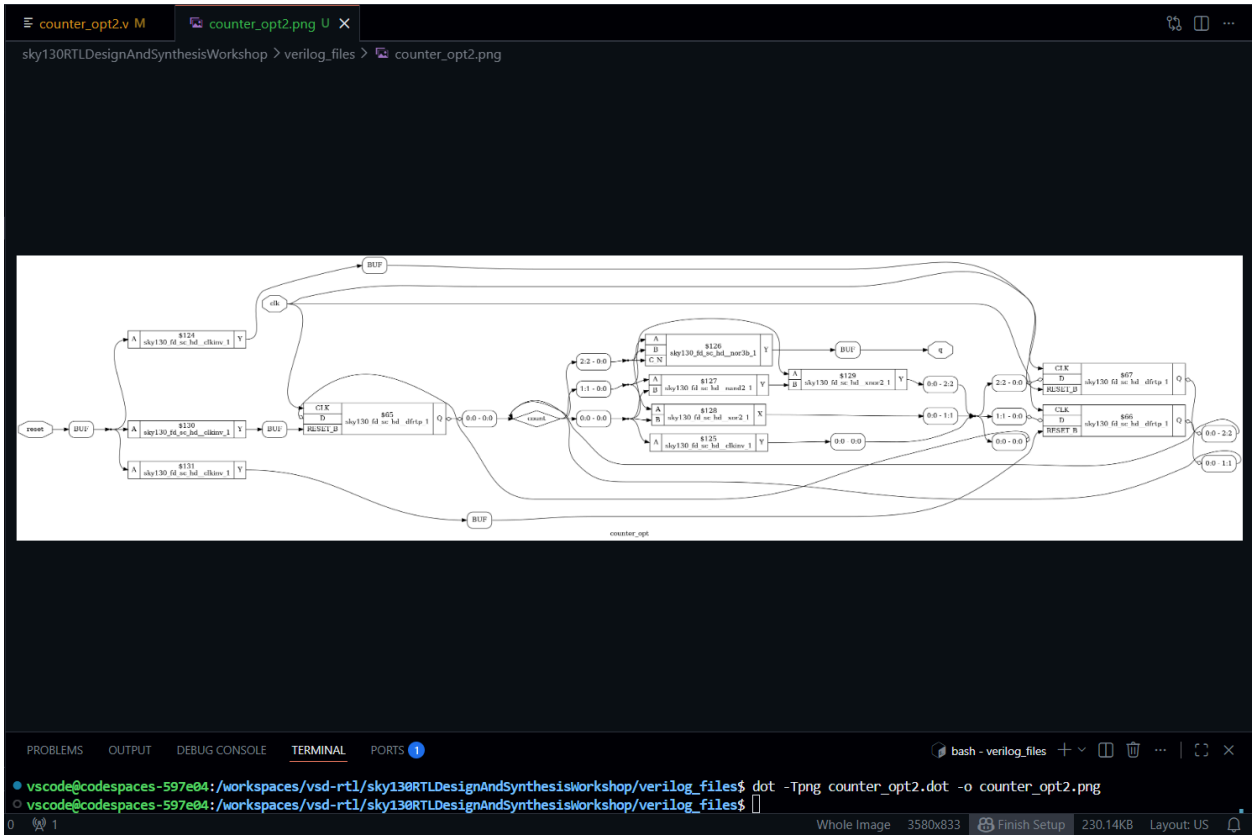


This set of images represents the simulation and synthesized implementation of the dff_const4 design, where constant logic values are directly driving the outputs. From the GTKWave waveform, it is observed that both outputs (q and q1) remain at a constant logic high (1) regardless of clock activity, since the inputs to these nodes are fixed at 1'b1. The reset signal does not affect the outputs in this case because the design has been optimized during synthesis. In the gate-level diagram, this optimization is clearly visible as the flip-flop elements are completely removed, and the outputs are driven directly through buffer cells connected to constant logic values. This indicates that the synthesis tool has recognized the constant behavior and simplified the circuit by eliminating unnecessary sequential elements, resulting in a purely combinational and highly optimized hardware implementation.



This set of images represents the simulation and synthesized gate-level implementation of the dff_const5 design using the SKY130 standard cell library. The GTKWave waveform shows the behavior of signals such as clk, reset, intermediate signal q1, and final output q. Initially, when reset is active, the outputs are held at a known state, and once reset is deasserted, the first flip-flop captures the constant input (1'b1) on the clock edge, producing q1. This intermediate signal is then fed into a second flip-flop, introducing a one-clock-cycle delay before appearing at the final output q, which is clearly visible in the waveform. In the synthesized diagram, this behavior is implemented using two cascaded D flip-flops (sky130_fd_sc_hd__dfrtp_1) along with buffer and clock inverter cells to maintain proper signal integrity and polarity. The presence of the constant input simplifies part of the logic, but the sequential nature is preserved due to the cascading of flip-flops. This confirms correct functional behavior and accurate hardware mapping of a pipelined flip-flop structure.

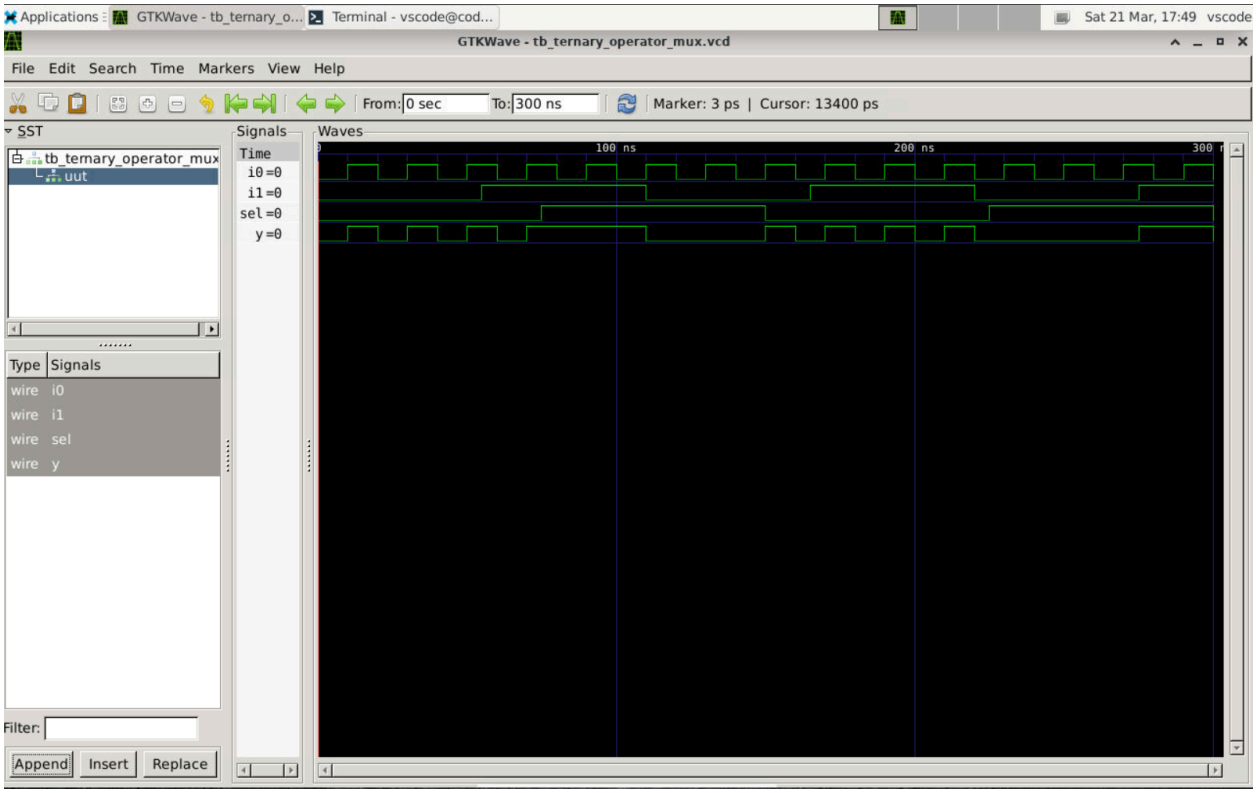


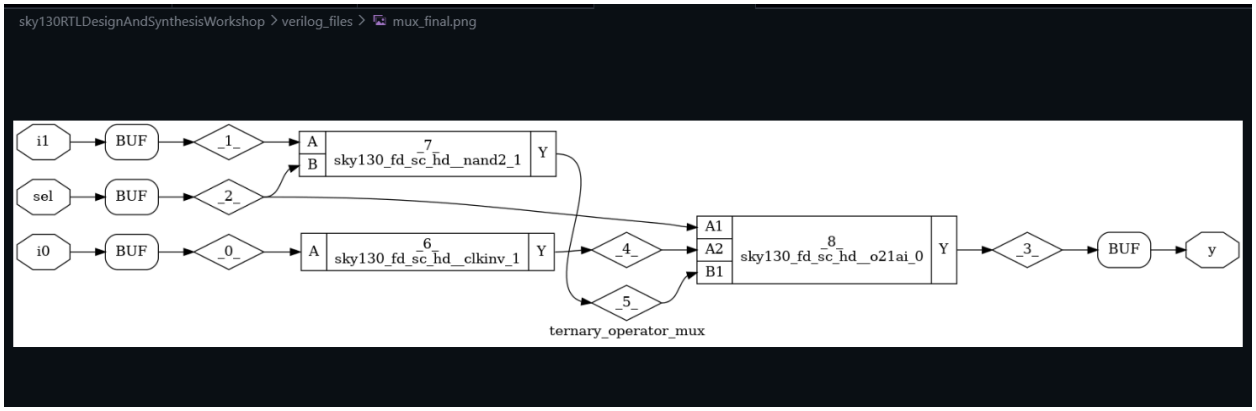


These images represent the synthesized and optimized gate-level implementation of a counter design using the SKY130 standard cell library. In the simpler version (counter_opt), the design consists of a D flip-flop (sky130_fd_sc_hd__dfrtp_1) with reset control, along with clock inverter and buffer cells to maintain proper signal integrity. The flip-flop stores the current state (count), and the feedback path updates the next state based on the counting logic, forming a sequential circuit. In the more complex optimized version (counter_opt2), additional combinational logic blocks such as NAND, NOR, and XOR gates are introduced to implement multi-bit counting behavior. These gates generate the next-state logic based on the current outputs, while multiple flip-flops store each bit of the counter. Buffers and clock inverter cells are used throughout to ensure proper timing and signal strength. The interconnections show feedback loops typical of

counter circuits, where outputs are fed back as inputs to compute the next count value. Overall, these diagrams confirm that the RTL counter design has been successfully synthesized, optimized, and mapped into a hardware-level implementation using standard cells, demonstrating both sequential storage and combinational next-state logic.

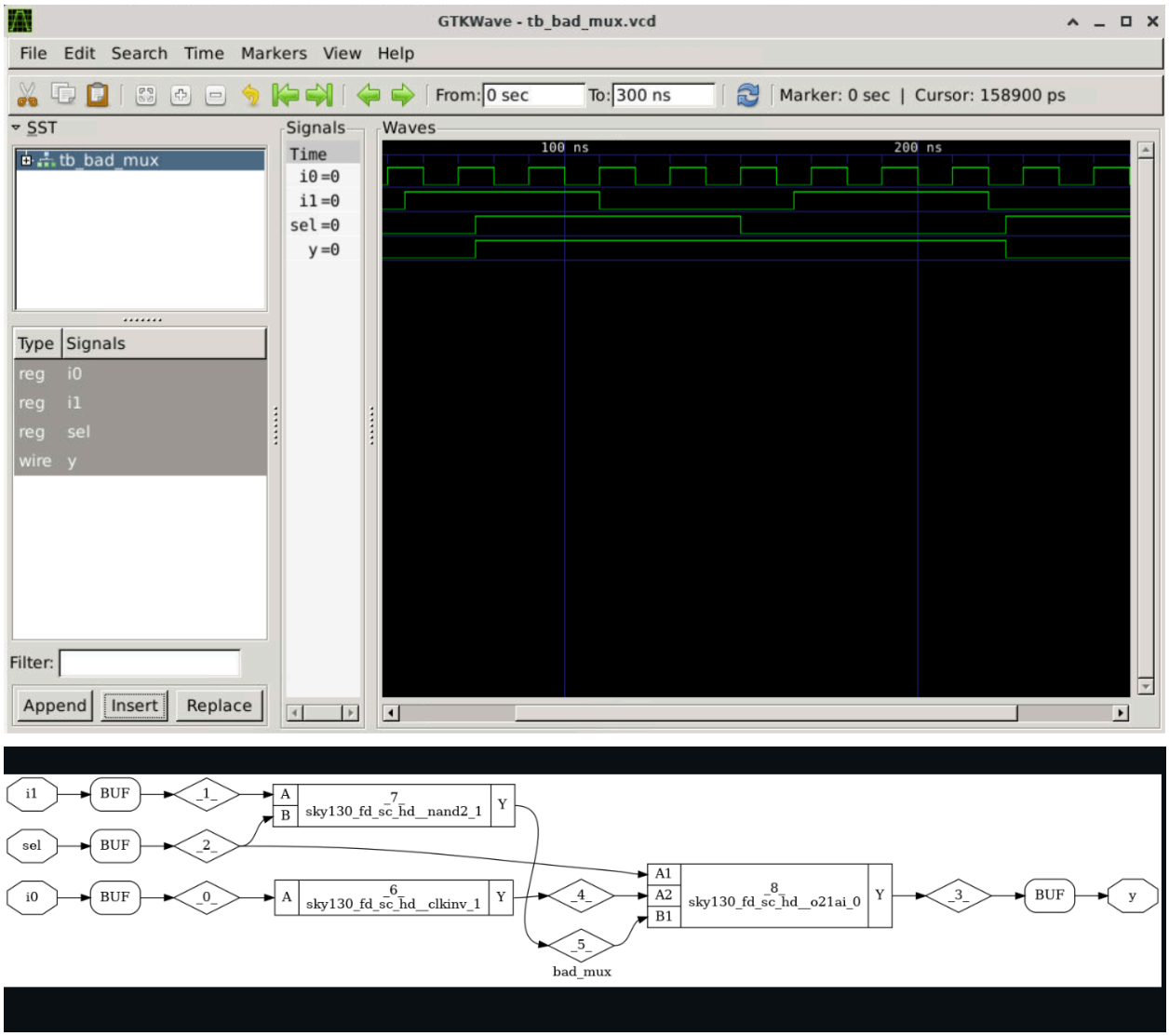
Day 4:



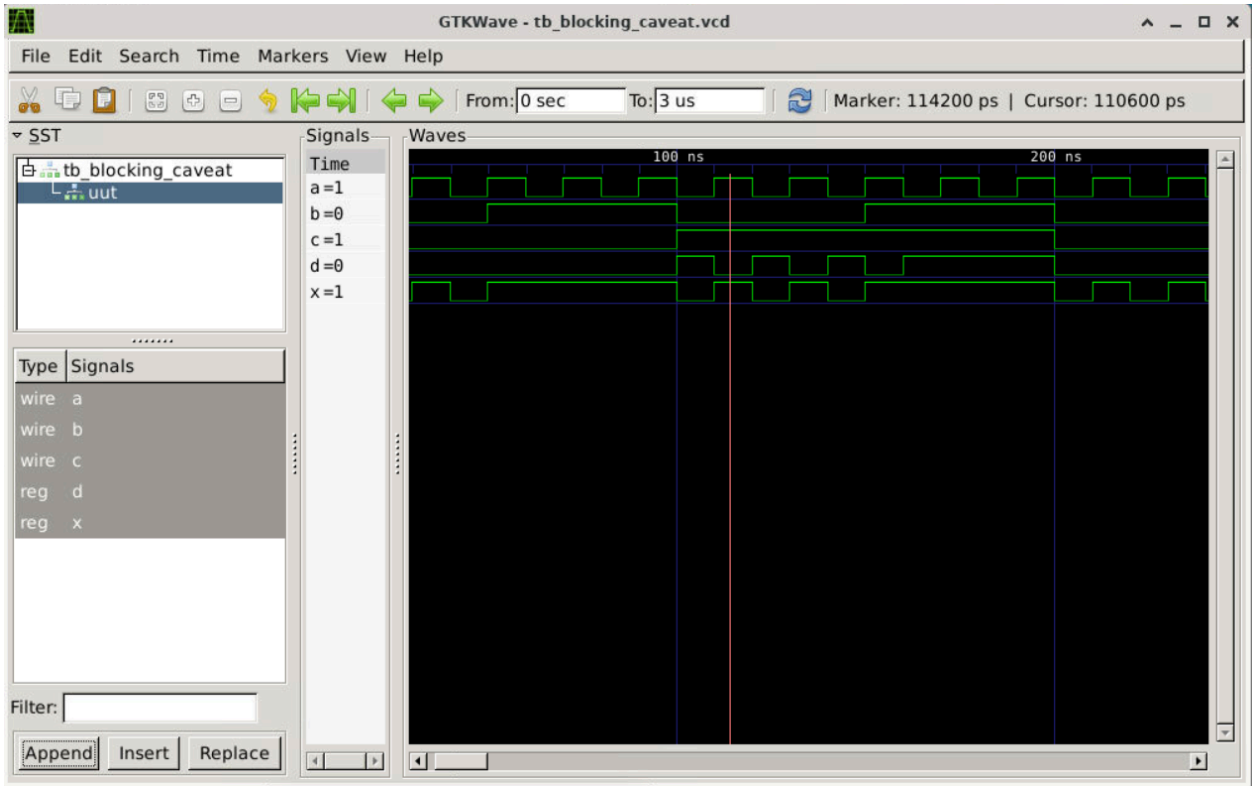


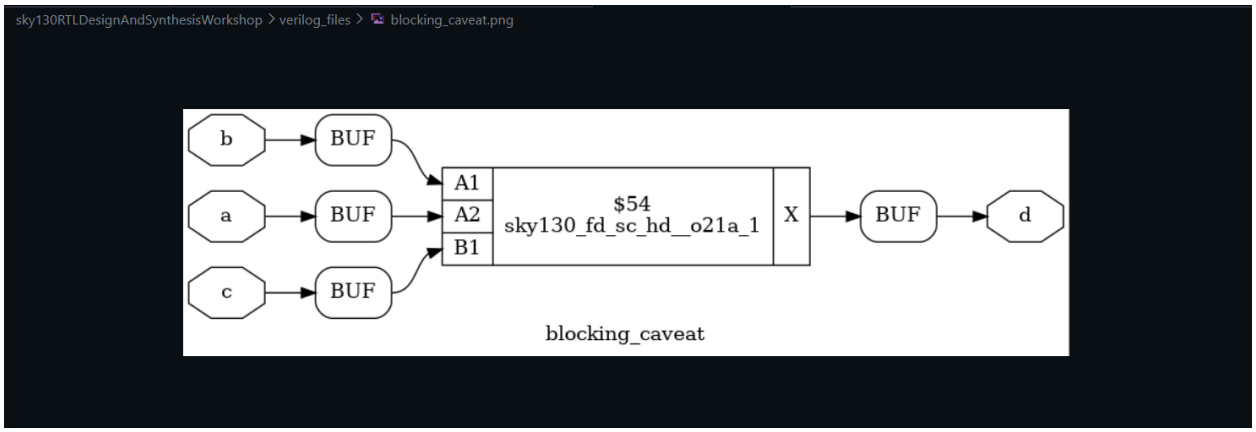
This set of images represents the simulation and synthesized gate-level implementation of a 2:1 multiplexer designed using a ternary operator. The GTKWave waveform shows the behavior of signals i0, i1, sel, and output y, confirming correct functionality: when sel = 0, the output follows i0, and when sel = 1, the output follows i1. This verifies the proper operation of the ternary expression used in the RTL design. In the synthesized gate-level diagram, the multiplexer is implemented using SKY130 standard cells such as NAND gates, clock inverters, and complex logic cells like o21ai, along with buffer cells to maintain signal integrity and drive strength. Instead of using a single dedicated MUX cell, the synthesis tool has optimized the design into a combination of logic gates that perform the same selection function. The interconnection of these gates reflects the Boolean implementation of the multiplexer logic, where the select signal controls which input is propagated to the output. Overall, these results confirm that the RTL

ternary operator has been correctly synthesized into an efficient hardware-level implementation using standard cells, with matching functional behavior observed in the waveform.



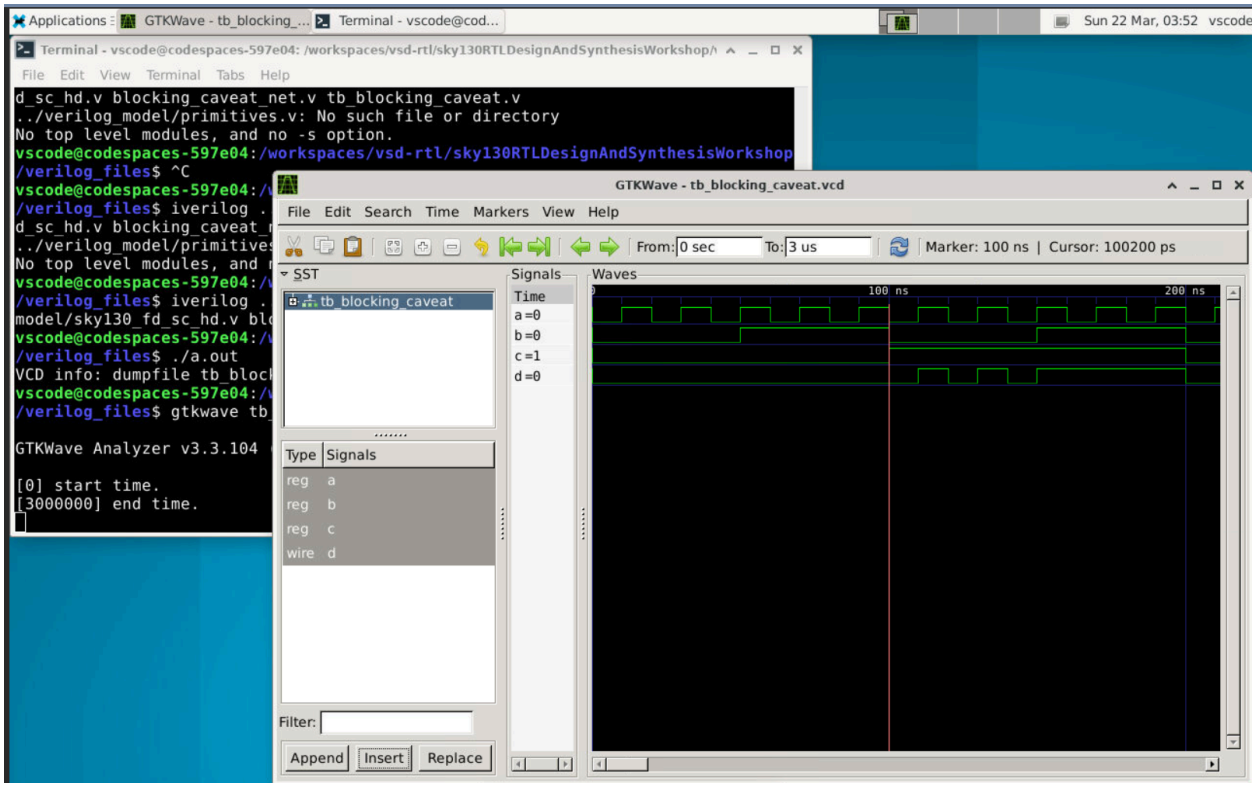
This set of images represents the simulation and synthesized gate-level implementation of an incorrectly designed multiplexer (bad_mux). The GTKWave waveform shows that the output y does not consistently follow the expected behavior of a proper 2:1 MUX, as it does not reliably switch between i0 and i1 based on the select signal sel. This indicates a flaw in the RTL description, leading to incorrect logic behavior. In the synthesized diagram, the design is implemented using SKY130 standard cells such as NAND gates, clock inverter cells, and a complex gate (o21ai), along with buffers for signal integrity. However, unlike an optimized MUX implementation, the logic structure does not correctly represent the standard multiplexing equation, resulting in improper signal selection. The interconnections reveal that the select signal is not correctly controlling the data paths, which explains the mismatched waveform output. This demonstrates how an incorrect RTL description can still be synthesized into hardware, but with unintended functionality, highlighting the importance of proper design and verification before synthesis.



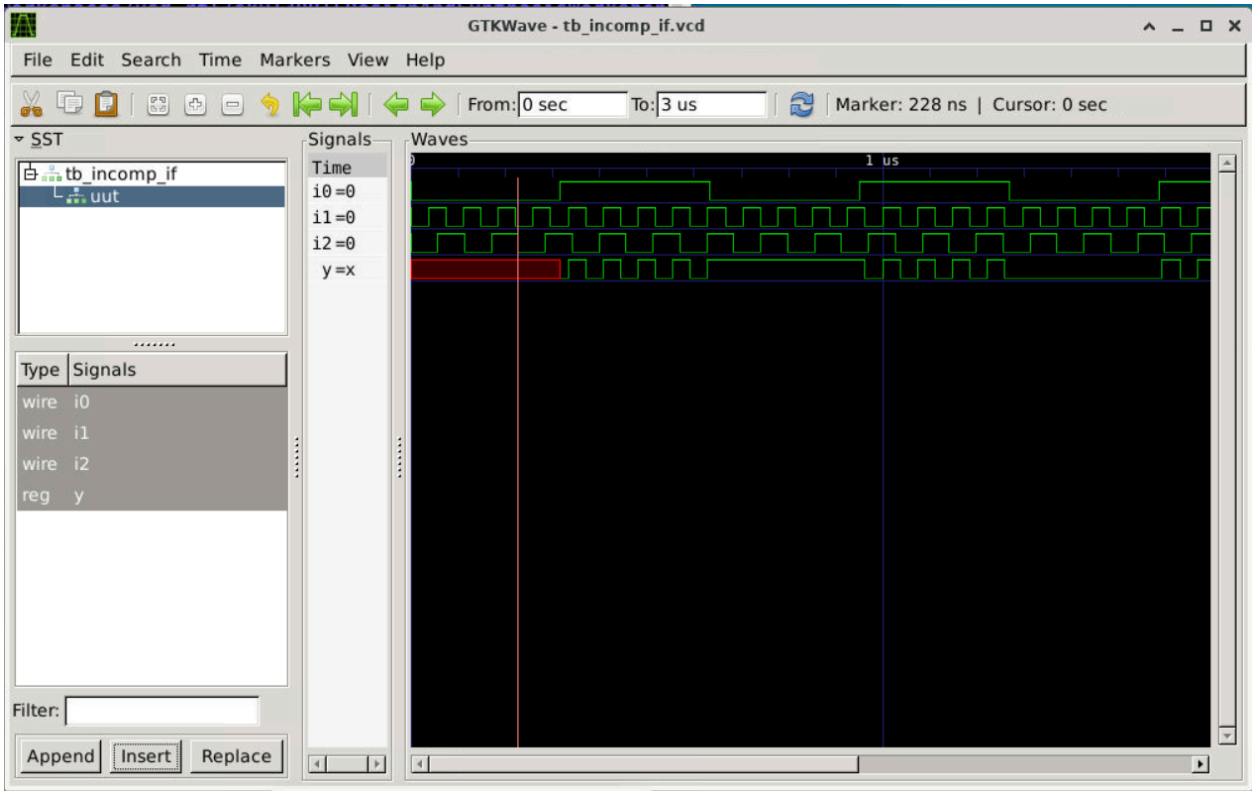


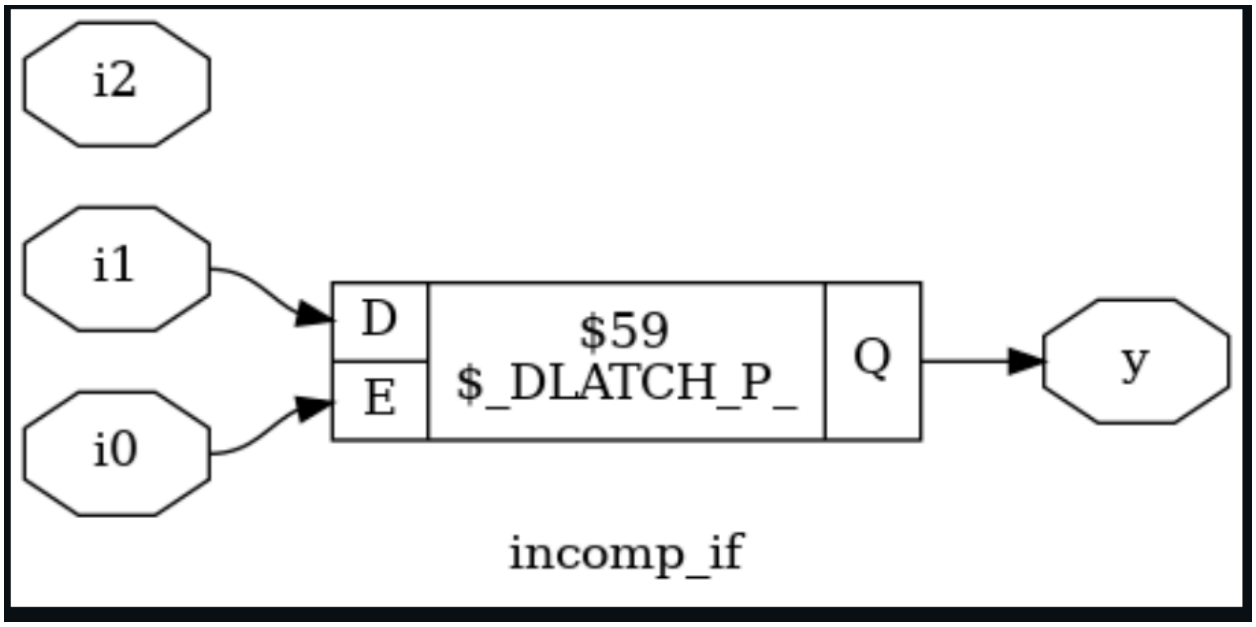
This set of images represents the simulation and synthesized gate-level implementation of a design demonstrating the blocking assignment caveat in Verilog. The GTKWave waveform shows signals a, b, c, intermediate x, and output d, where the behavior reflects sequential execution within a single always block due to blocking (=) assignments. As a result, changes in one variable immediately affect subsequent statements, leading to unintended behavior compared to parallel (non-blocking) execution. This is visible in the waveform where x and d do not behave as independently updated signals but instead show dependency on the order of assignments. In the synthesized diagram, the logic is implemented using a SKY130 complex gate (sky130_fd_sc_hd__o21a_1) along with buffer cells, combining inputs a, b, and c into a single combinational expression. The synthesis tool interprets the final logic equation rather than the step-by-step procedural behavior, resulting in a compact hardware implementation. This highlights the key difference between simulation semantics and synthesized hardware,

emphasizing the importance of using non-blocking assignments (<=) in sequential logic to avoid unintended results.



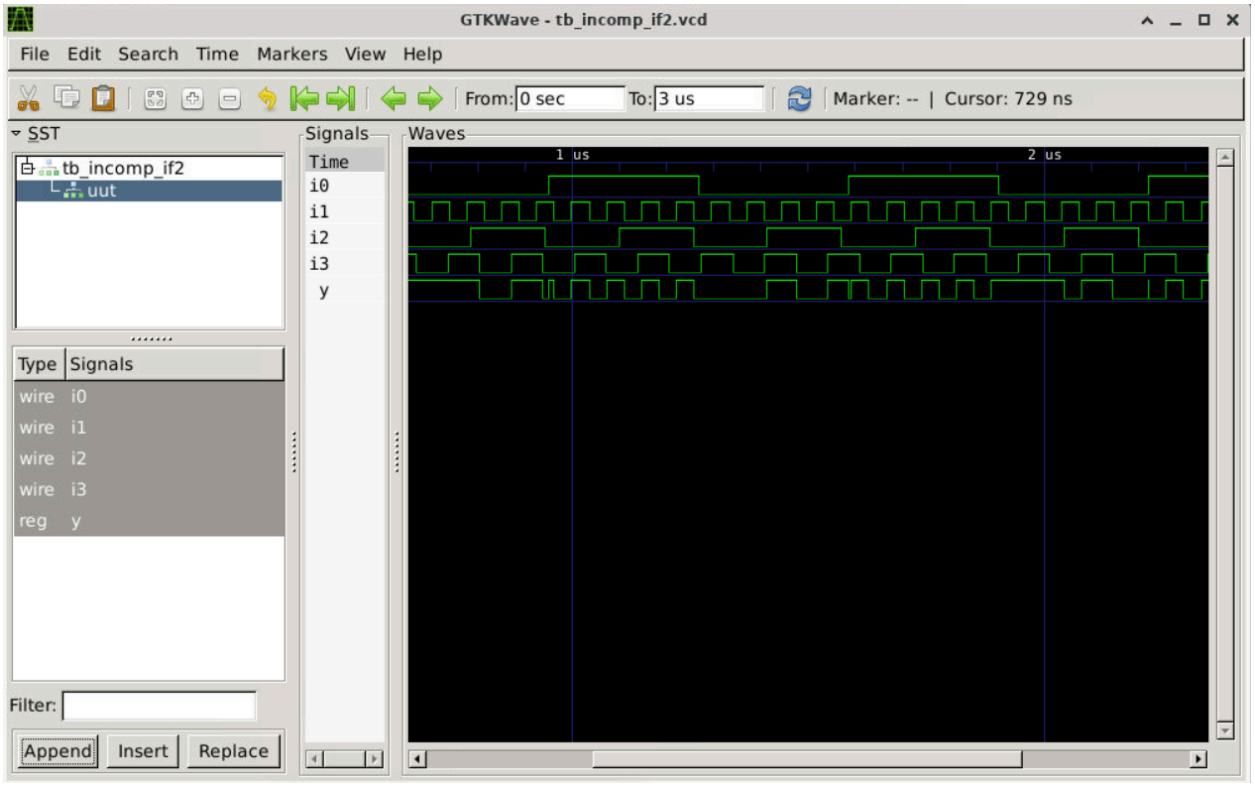
Day 5:

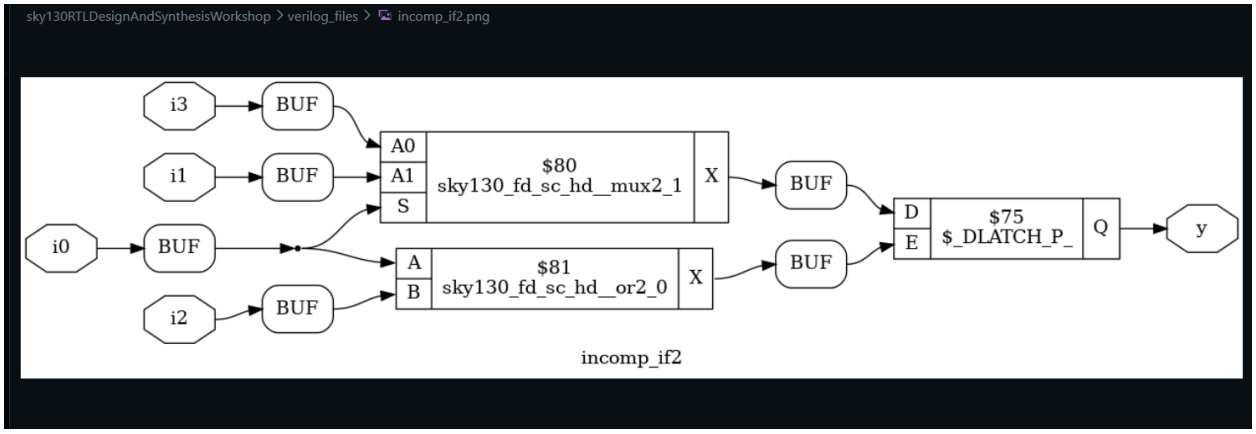


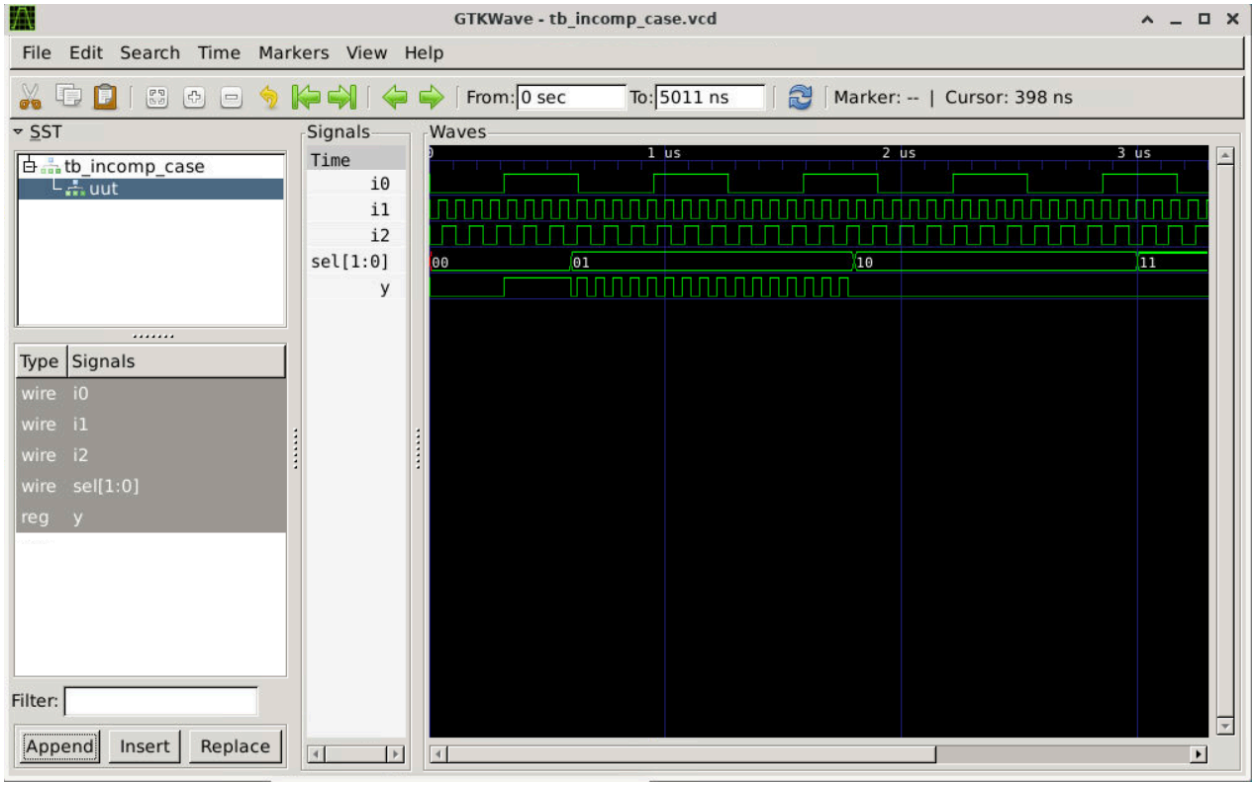


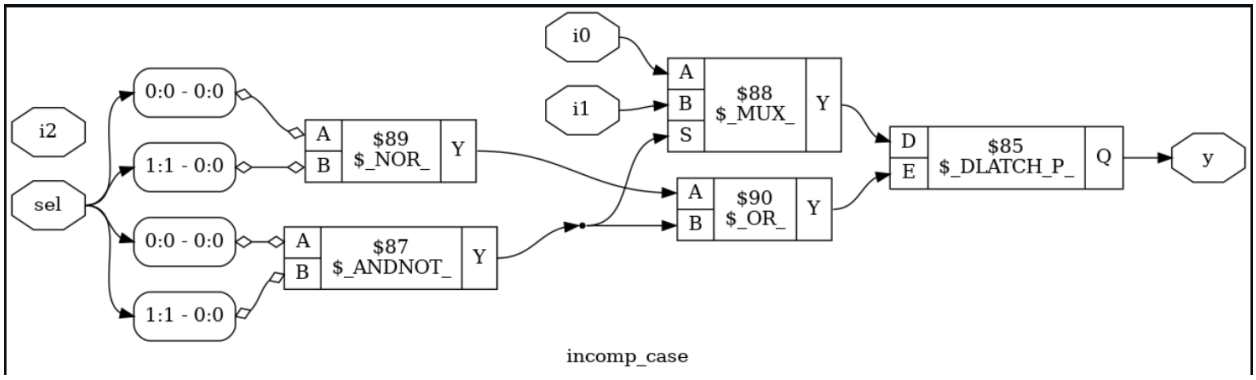
This set of images represents the simulation and synthesized gate-level implementation of a design with an incomplete if statement (incomp_if). The GTKWave waveform shows signals i0, i1, i2, and output y, where the output initially goes to an unknown (x) state and then retains previous values under certain conditions. This behavior occurs because not all conditions are specified in the RTL code, leading to memory-like behavior. In the synthesized diagram, this is

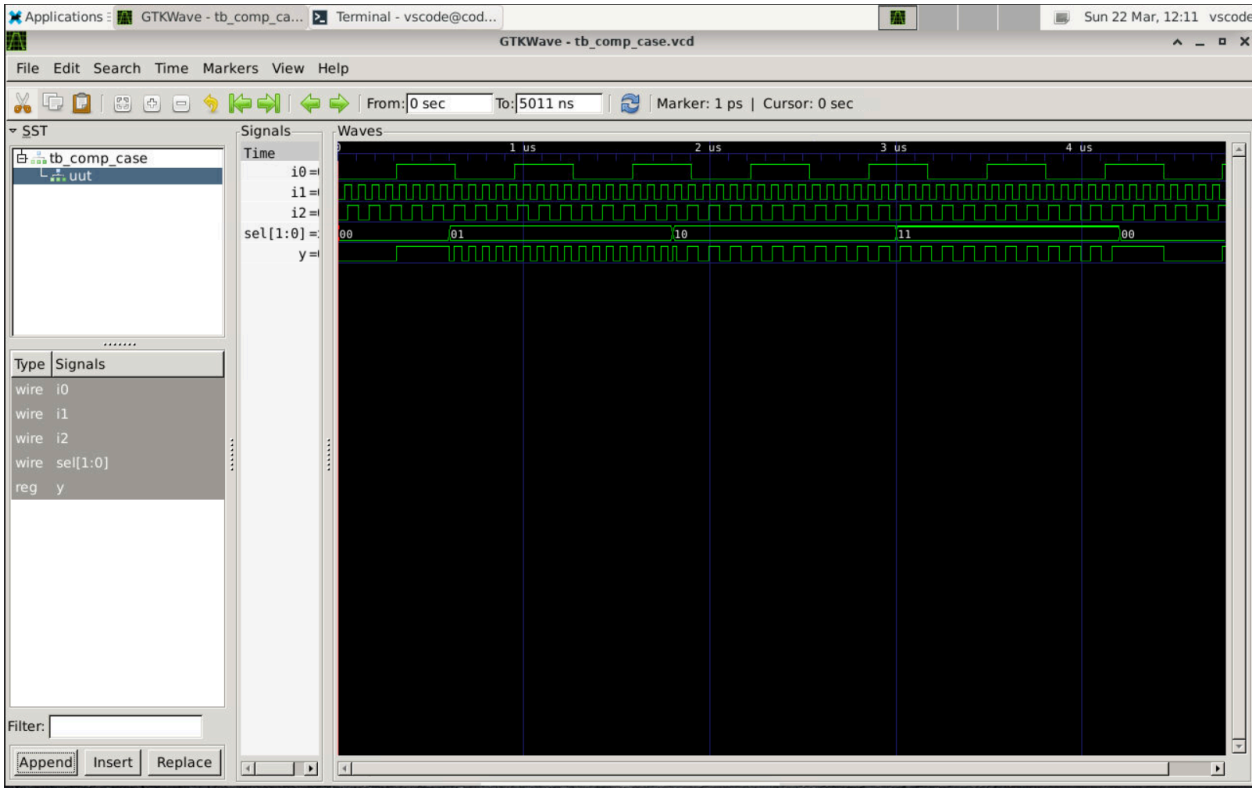
implemented using a latch ($_DLATCH_P_), where i1 is connected to the data input (D) and i0 acts as the enable signal (E). Since the if condition is incomplete, the synthesis tool infers a latch to hold the previous value of y when the condition is not satisfied. This demonstrates how missing conditions in combinational logic can unintentionally create sequential elements like latches, highlighting the importance of writing complete if-else statements to avoid unintended hardware inference.

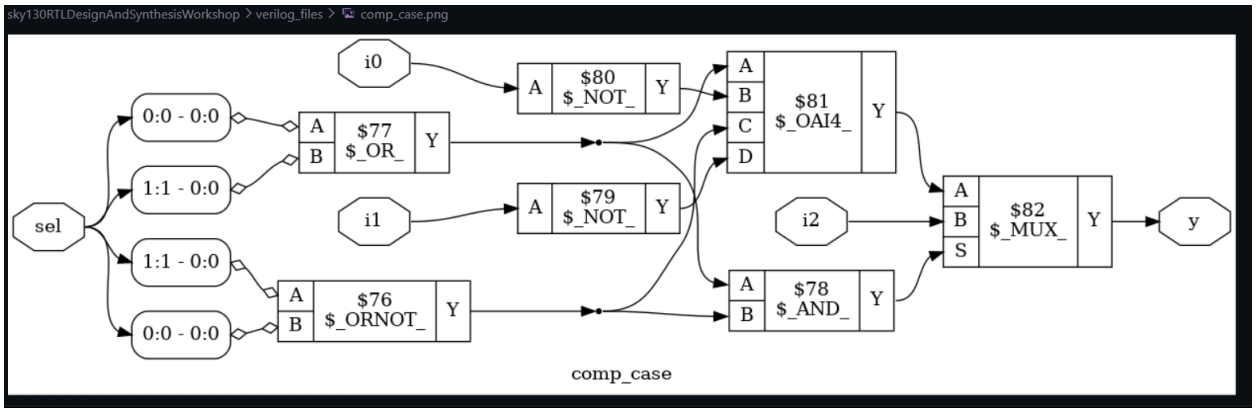




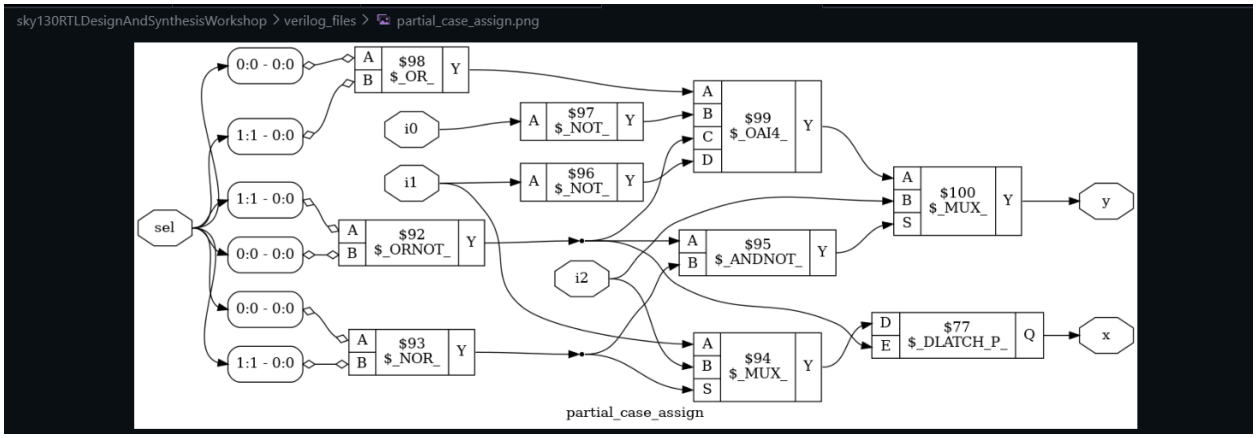

GTKWave - tb_incomp_case.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 5011 ns
Marker: -- | Cursor: 398 ns
SST
tb_incomp_case
uut
Type Signals
wire i0
wire i1
wire i2
wire sel[1:0]
reg y
Filter:
Append Insert Replace
Signals
Time
i0
i1
i2
sel[1:0]
y
Waves
1 us
2 us
3 us
00
01
10
11


i2
sel
0:0 - 0:0
1:1 - 0:0
0:0 - 0:0
1:1 - 0:0
A
B
$89
$_NOR_
Y
A
B
$87
$_ANDNOT_
Y
i0
i1
A
B
S
$88
$_MUX_
Y
A
B
$90
$_OR_
Y
D
E
$85
$_DLATCH_P_
Q
y
incomp_case

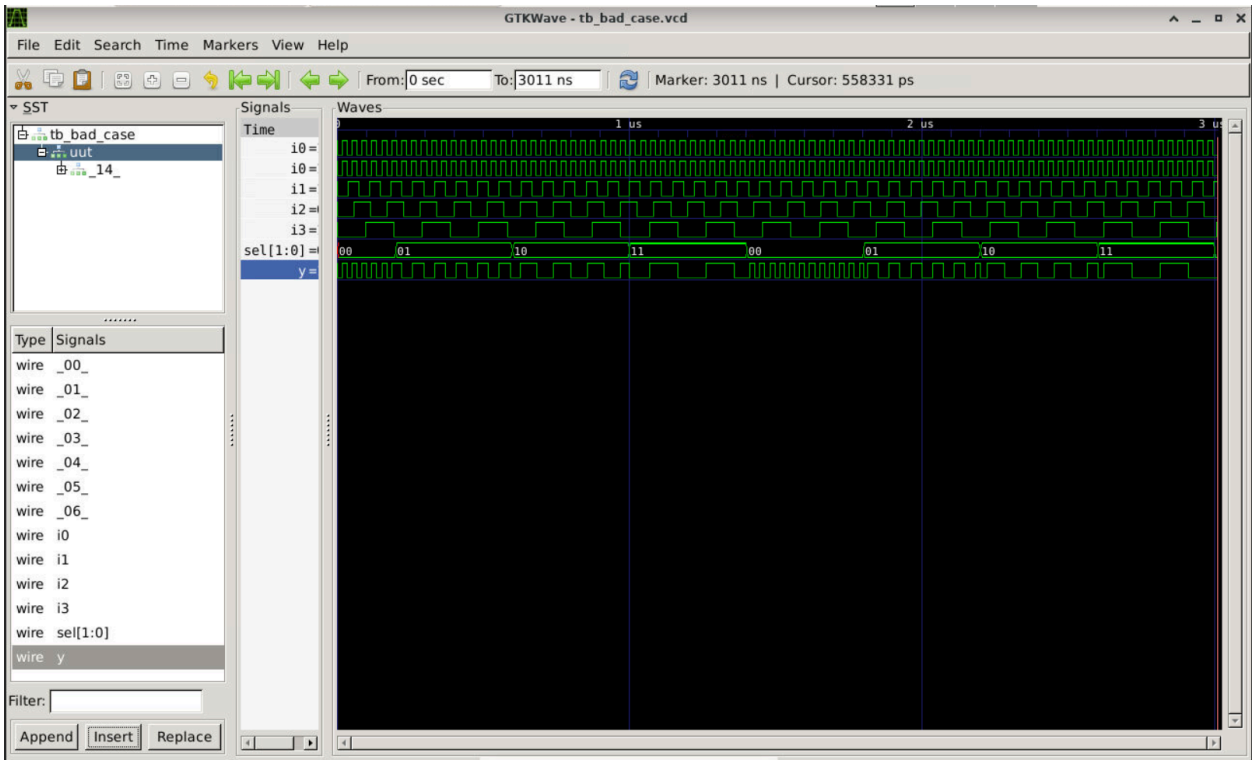




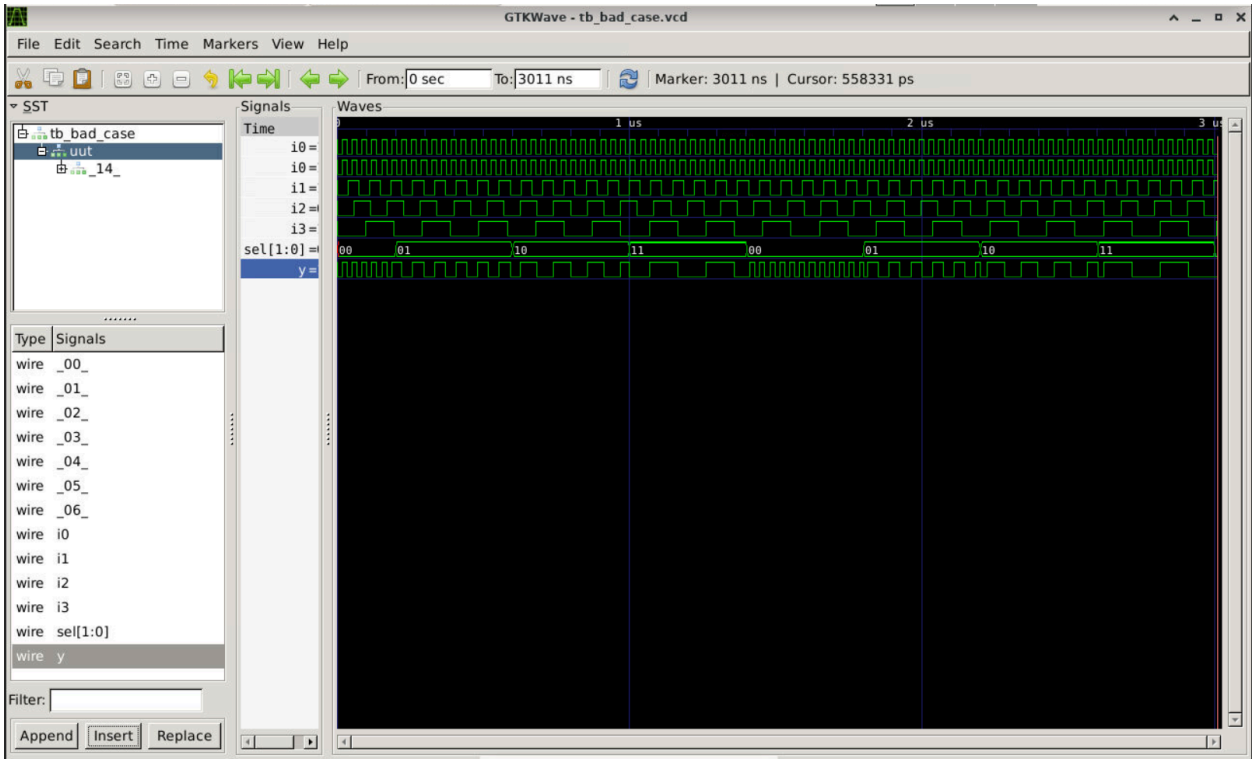
The figure shows the synthesized hardware implementation of a Verilog case statement, where high-level behavioral logic is converted into gate-level circuitry. The select signal sel is decoded into different conditions that drive combinational logic blocks such as $OR, $AND, $NOT, and $OAI4. Inputs i0, i1, and i2 pass through these logic elements, and their processed outputs are fed into a final multiplexer ($MUX). This multiplexer selects the appropriate signal based on sel, producing the output y. Overall, the diagram demonstrates how a case statement is realized in hardware as a combination of logic gates followed by a mux for final selection.



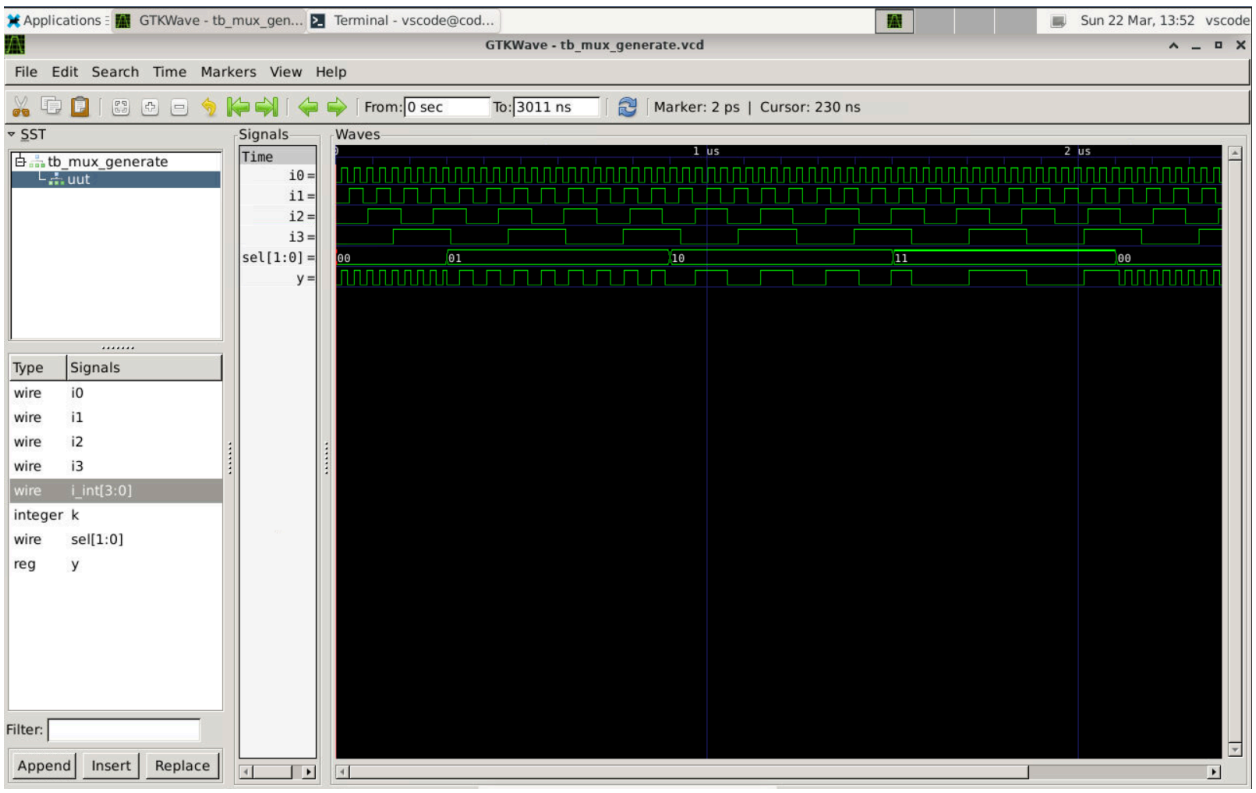
No latch in path of y but there is a latch in path of x



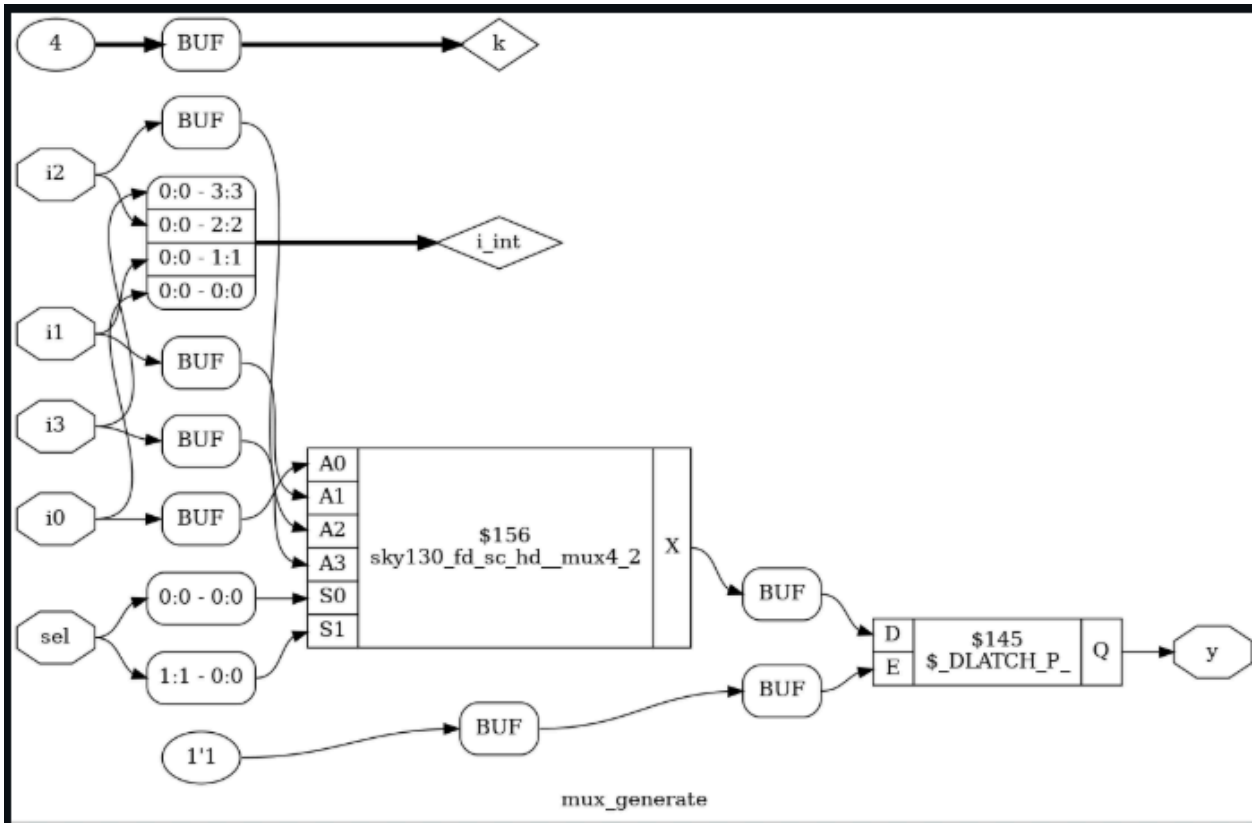
Netlist:



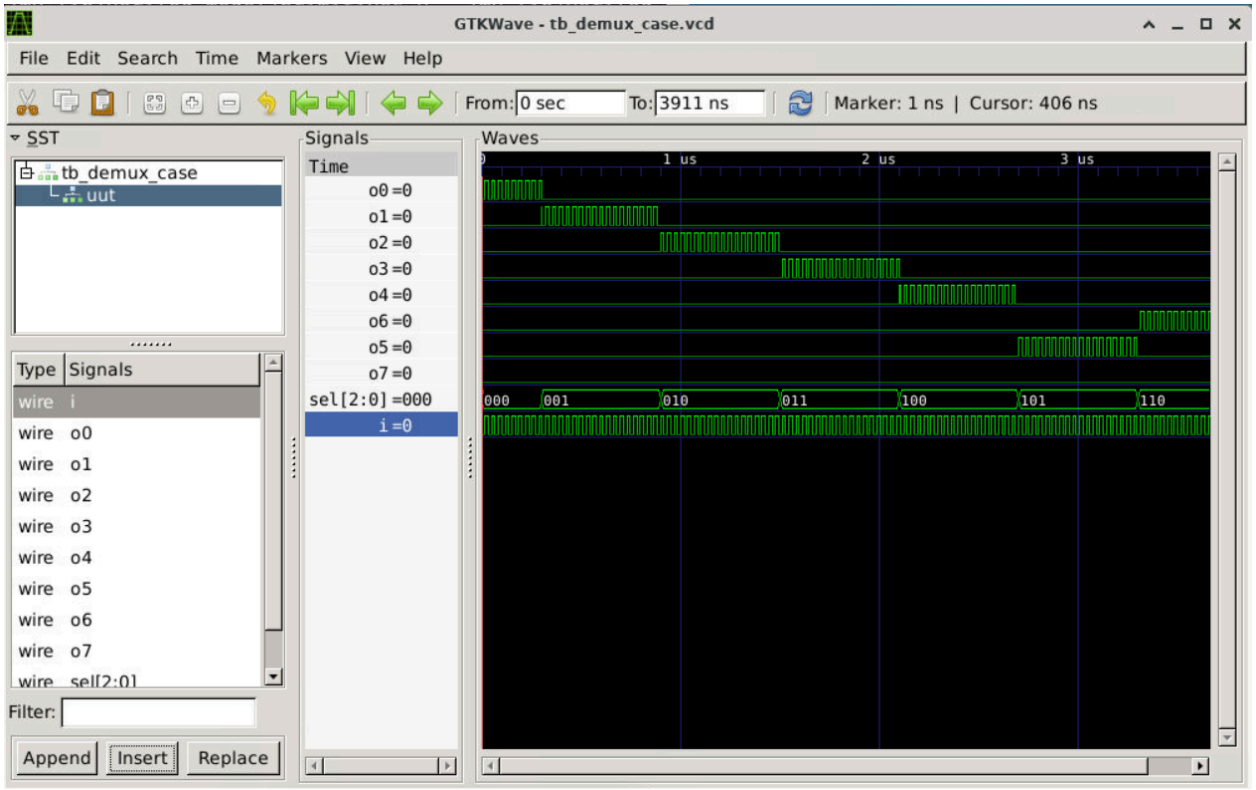
Mux using for loop



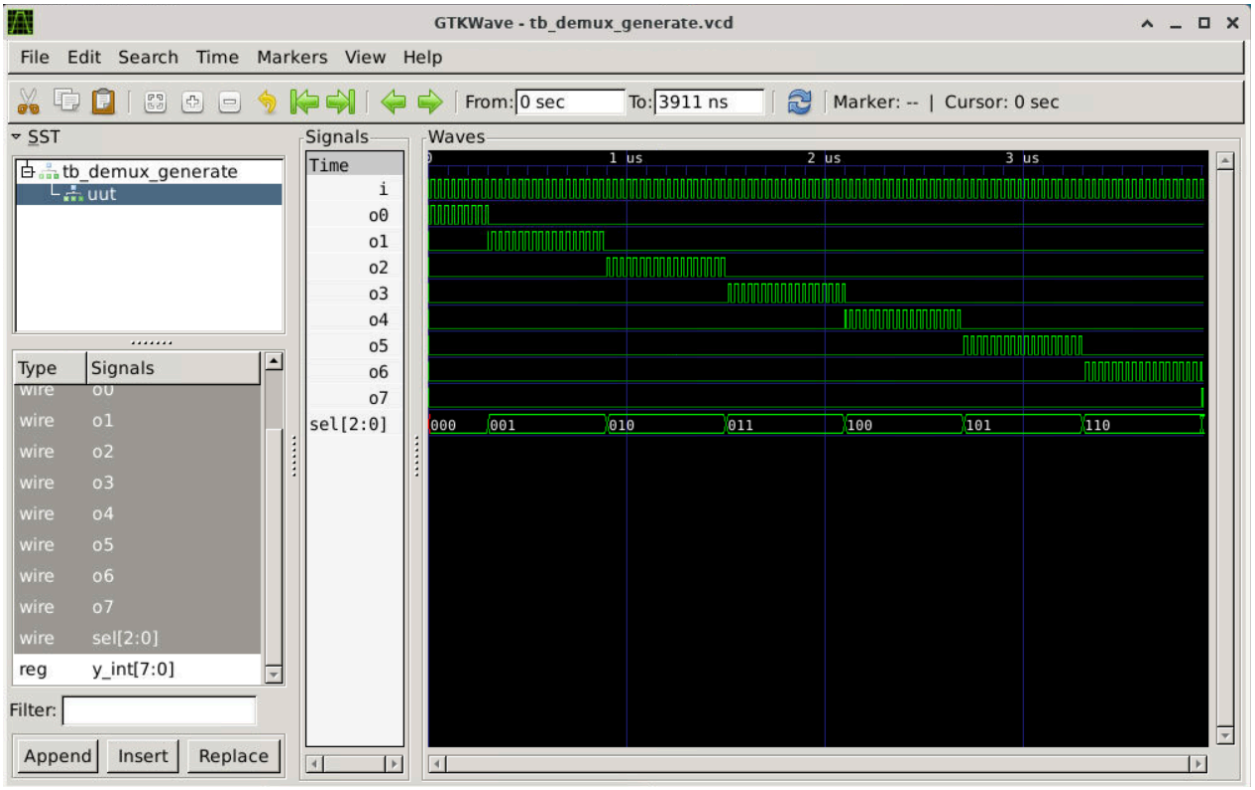
This is 4:1 mux



Demux 8:1 case:



Demux 8:1 generate:



Ripple Adder: